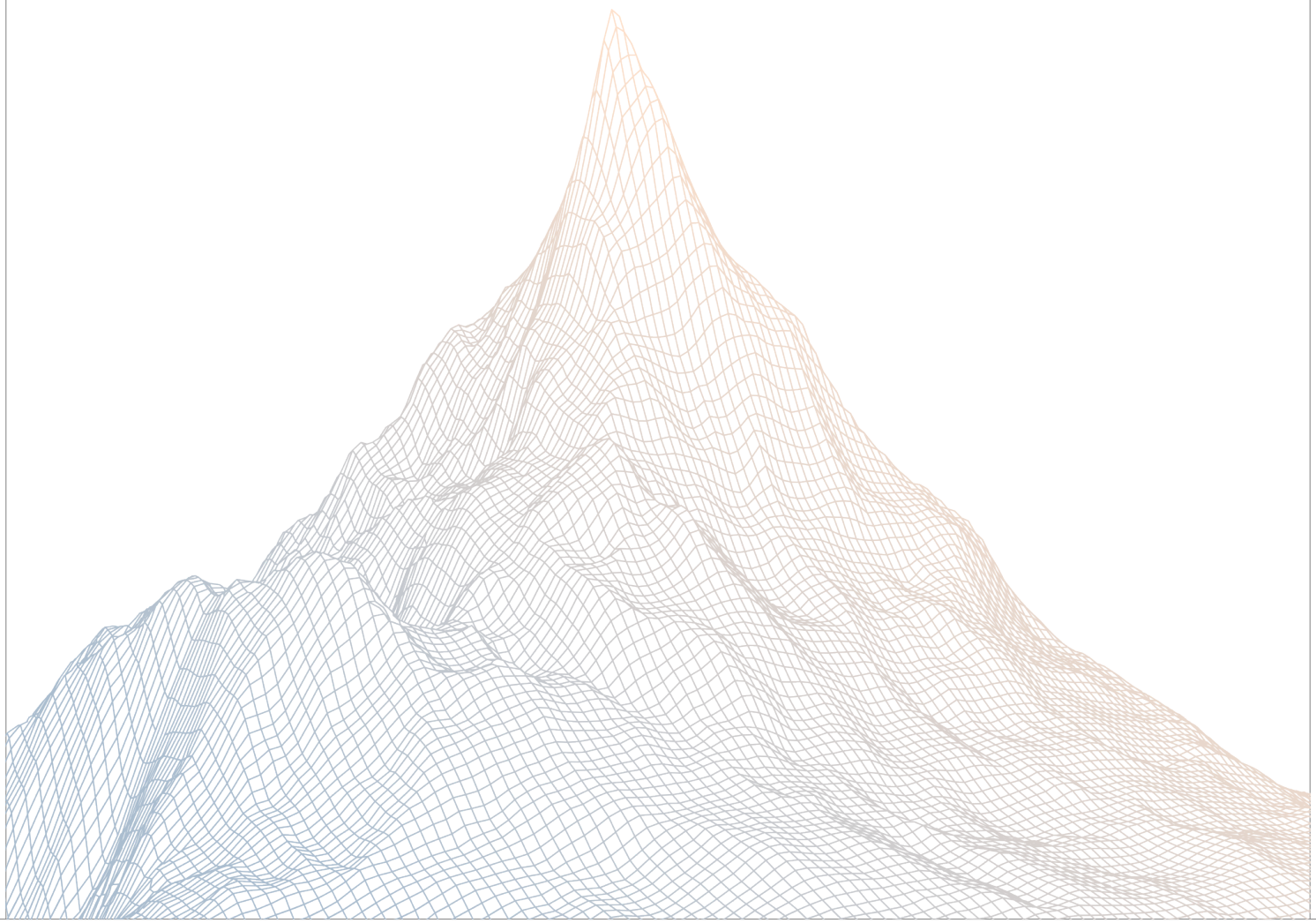
Zenith

Daimon DAO

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: August 10th to August 13th, 2026

AUDITED BY: Mario Poneder
adriro

Contents

1 Introduction 3

1.1 About Zenith 3

1.2 Disclaimer 3

1.3 Risk Classification 3

2 Executive Summary 4

2.1 About Daimon DAO 4

2.2 Scope 4

2.3 Audit Timeline 6

2.4 Issues Found 6

3 Findings Summary 7

4 Findings 10

4.1 Critical Risk 10

4.2 High Risk 13

4.3 Medium Risk 15

4.4 Low Risk 39

4.5 Informational 74

1 Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at https://zenith.security.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Daimon DAO

Daimon is a BEP-20 token with reflection, vote-escrow staking, on-chain governance and a public timelock, on BNB Chain / PancakeSwap. Control belongs to the DAO through a Timelock; the deployer renounces every role after deploy. The supply can only decrease, never below an immutable floor. Everything the contracts do is verifiable on-chain, and the full source is public.

2.2 Scope

The engagement involved a review of the following targets:

Target	daimon-dao
Repository	https://github.com/daimon-dao/daimon-dao
Commit Hash	`4f89ab09d878b6d94e876c52be9a4920d4e8620f`

Files	`DaimonGovernor.sol`
	`DaimonMigration.sol`
	`DaimonStaking.sol`
	`DaimonTimelock.sol`
	`DaimonV2.sol`

Target	Mitigation Review
Repository	https://github.com/daimon-dao/daimon-dao
Commit Hash	`87ac723762cfc17d4ce3b58f1f52f1f37e99775a`

Files	`DaimonGovernor.sol`
	`DaimonMigration.sol`
	`DaimonStaking.sol`
	`DaimonTimelock.sol`
	`DaimonV2.sol`

2.3 Audit Timeline

August 10, 2026	Audit start
August 13, 2026	Audit end
August 18, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	1
High Risk	1
Medium Risk	7
Low Risk	12
Informational	16
Total Issues	**37**

3

Findings Summary

ID	Description	Status
C-1	Same-timestamp staking injects post-snapshot voting power	Resolved
H-1	Mutable quorum retroactively changes completed proposal outcomes	Resolved
M-1	Automatic fee sales can under-mint liquidity additions	Resolved
M-2	Uncaught automation failures can block sells and liquidity bootstrap	Resolved
M-3	Dust transfers bypass aggregate automation pacing	Resolved
M-4	Reflections accrued by the AMM pair can be permissionlessly skimmed	Resolved
M-5	Reflection accrued by Staking is unrecoverable through current interfaces	Resolved
M-6	A minimal first stake can atomically capture the zero-staker reward backlog	Resolved
M-7	Guardian can indefinitely censor governance and preserve a global pause	Resolved
L-1	Unbounded migration duration can make claims or final settlement unusable	Resolved
L-2	Buyback BNB has no current-interface settlement path at the supply floor	Resolved
L-3	Native-BNB liquidity removal charges DMN fees twice without directly bounding final receipt	Resolved
L-4	Standard liquidity additions use gross DMN amounts despite taxed pair receipts	Resolved
L-5	An unbounded proposal threshold can permanently disable proposal creation	Resolved

ID	Description	Status
L-6	Unbounded Timelock delay can permanently disable new governance scheduling	Resolved
L-7	Permissionless pair pre-creation can deny token proxy initialization	Resolved
L-8	Governor and Timelock cancellation state can diverge	Resolved
L-9	Migration instructions exempt the wrong address from predecessor-token fees	Resolved
L-10	Unbounded reward index can overflow per-account reward accounting	Resolved
L-11	Mutable mandatory fee exemptions can corrupt staking accounting and disrupt migration	Resolved
L-12	Spot-derived minimum output does not provide the claimed MEV-loss bound	Resolved
I-1	Staking assumes deposits are received in full	Resolved
I-2	A taxed final migration sweep permanently closes recovery after an incomplete transfer	Resolved
I-3	Caught router failures leave residual DMN allowances	Resolved
I-4	Staking governance changes lack a dedicated event	Resolved
I-5	Timelock accepts cancellation of nonexistent operations	Resolved
I-6	Unswapped fee inventory adopts the current marketing-to-buyback ratio	Resolved
I-7	Zero-value transfers and token events are not fully standards-compatible	Resolved

ID	Description	Status
I-8	Governor state omits queued status and treats unknown proposals as defeated	Resolved
I-9	Governance approvals do not expire automatically	Resolved
I-10	EXECUTOR_ROLE cannot be opened as documented	Resolved
I-11	A predecessor operation can appear as executed before its call finishes	Resolved
I-12	Emergency pause does not stop real supply burns	Resolved
I-13	Treasury custody permits repeated migration of legacy tokens	Resolved
I-14	Duplicate queue attempts rely on Timelock reverts	Resolved
I-15	Guardian can permanently pre-cancel proposals that do not yet exist	Resolved
I-16	Plain BNB transfers to staking are accepted but never accounted	Resolved

4

Findings

4.1 Critical Risk

A total of 1 critical risk findings were identified.

[C-1] Same-timestamp staking injects post-snapshot voting power

SEVERITY: Critical	IMPACT: High
STATUS: Resolved	LIKELIHOOD: High

Target

- DaimonGovernor.sol

Description:

A proposal snapshots `totalVotingPower` and `block.timestamp` before returning. Staking checkpoints use the same timestamp and `votingPowerAt()` accepts the last checkpoint whose timestamp is less than or equal to the proposal timestamp.

Consequently, voting power acquired after `propose()` but later in the same transaction or block is:

- absent from `snapshotTotalVotingPower`, but
- included in the attacker's `votingPowerAt(attacker, snapshotTimestamp)`.

The same-timestamp checkpoint logic either creates a checkpoint at the proposal timestamp or overwrites an existing checkpoint at that timestamp. Transaction ordering—not merely block ordering—is therefore erased.

This can be exploited atomically through a coordinator contract, it does not require reacting to an already-mined proposal or controlling a validator. An attacker can introduce voting weight after seeing and creating the proposal while omitting that weight from the quorum denominator. If the injected weight clears quorum and exceeds opposing votes, the attacker can pass an arbitrary Timelock action, including a UUPS upgrade capable of taking control of the full token state.

Under current defaults, one maximum-size 365-day stake grants:

- `5 billion DMN × 4 = 20 billion` voting-power units.

Multiple stakes can be included in the same transaction or block, bounded by the attacker's

liquid balance and block gas. The maximum default theoretical injected weight is approximately four times the liquid supply.

The attack still requires real DMN to be locked and retains the normal voting and Timelock periods, but it defeats the protocol's central promise that power obtained after proposal creation cannot affect that proposal.

Proof of Concept:

```
function test_PoC_sameBlockStakeBypassesSnapshot() public {
    address attacker = address(0xBAD);

    _stake(address(0xBEEF), 99_000 ether, 0);
    _stake(attacker, 1_000 ether, 0); // enough to propose
    fundWithDmn(attacker, 9_000 ether);
    assertEq(staking.totalVotingPower(), 100_000 ether);

    vm.startPrank(attacker);
    token.approve(address(staking), 9_000 ether);
    uint256 id = governor.propose(address(token), 0, "", "p");
    uint256 snapshot = block.timestamp;
    staking.stake(9_000 ether, 0); // no warp: checkpoint is written at the
    snapshot
    vm.stopPrank();

    assertEq(staking.votingPowerAt(attacker, snapshot), 10_000 ether);
    assertEq(staking.totalVotingPower(), 109_000 ether); // consistent quorum:
     10,900 VP

    vm.warp(snapshot + governor.VOTING_DELAY());
    vm.prank(attacker);
    governor.castVote(id, 1);

    vm.warp(snapshot + governor.VOTING_DELAY() + governor.VOTING_PERIOD() + 1)
    ;
    assertEq(_state(id), SUCCEEDED); // passes against the stale 10,000 VP
    quorum
}
```

Recommendations:

Use checkpoints with ordering granularity that cannot be modified after the snapshot. Prefer block-number checkpoints for both per-account and aggregate voting power. When creating a proposal:

- set the snapshot to the last completed block, `block.number - 1`
- obtain voter weights from `votingPowerAt(account, snapshotBlock)`
- obtain the quorum denominator from `totalVotingPowerAt(snapshotBlock)`

If timestamp checkpoints are retained, using `block.timestamp - 1` prevents the specific post-proposal staking injection because subsequent checkpoints cannot affect that past timepoint. However, BSC can produce several adjacent blocks with the same whole-second timestamp, so this represents the previous second rather than precisely the previous block and may exclude legitimate voting-power changes from earlier blocks in the current second.

Daimon DAO: Resolved with `@e029db0 ...` — voting power checkpoints now keyed by block number rather than timestamp, and the proposal snapshot is taken at block.number - 1.

Zenith: Verified.

4.2 High Risk

A total of 1 high risk findings were identified.

[H-1] Mutable quorum retroactively changes completed proposal outcomes

SEVERITY: High	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Medium

Target

- src/DaimonGovernor.sol#L124-L138
- src/DaimonGovernor.sol#L164-L205
- src/DaimonGovernor.sol#L227-L233

Description:

The Governor snapshots total voting power when a proposal is created, but does not snapshot the applicable quorum percentage. Instead, `state()` evaluates every completed proposal using the current global `quorumBps`.

```
uint256 quorumNeeded =
    (p.snapshotTotalVotingPower * quorumBps) / 10000;
```

Consequently, changing `quorumBps` retroactively changes proposal outcomes. A proposal defeated solely for insufficient quorum can become `Succeeded` after quorum is lowered. Because unqueued proposals do not expire, anyone can subsequently queue the revived proposal and execute it after the Timelock delay.

Raising quorum causes the inverse failure. It can turn a successful or already queued proposal into `Defeated`. Although `queue()` sets `p.queued`, `state()` does not preserve a queued state and continues recalculating the result. `execute()` then requires the dynamically evaluated state to remain `Succeeded`, potentially stranding an operation already scheduled in the Timelock.

The provided deployment script initializes quorum at the 10% minimum. Revival therefore requires a reachable sequence such as raising quorum to 20%, creating a proposal that fails only under that threshold, and later lowering quorum to 10%. Raising quorum can affect

historical proposals on the first adjustment.

This does not bypass voting, queueing, or the seven-day Timelock. However, it breaks vote-outcome finality and can make an action rejected under its contemporaneous rules executable. Proposals support arbitrary targets, values, and calldata, so a revived proposal can perform governance-controlled upgrades, role or parameter changes, or transfers of Timelock funds. Guardian cancellation and the Timelock reaction window mitigate the risk but require continuous monitoring and do not prevent successful queued proposals from becoming non-executable.

Recommendations:

It is recommended to snapshot `quorumBps` for every proposal at creation and exclusively use that stored value when evaluating its outcome.

```
struct Proposal {
    // ...
    uint256 quorumBpsSnapshot;
}

p.quorumBpsSnapshot = quorumBps;

uint256 quorumNeeded =
    (p.snapshotTotalVotingPower * p.quorumBpsSnapshot) / 10000;
```

Alternatively, checkpoint quorum changes and query the value applicable to the proposal's snapshot, with same-timepoint behavior explicitly defined.

Add tests confirming that:

- lowering quorum does not revive defeated proposals;
- raising quorum does not defeat successful proposals;
- queued proposals remain executable after quorum changes; and
- newly created proposals use the updated quorum.

`DaimonGovernor` is not upgradeable. Existing deployments therefore require Governor replacement and Timelock role rewiring; the correction should be included before mainnet deployment.

Daimon DAO: Resolved with `@e029db0 ...`

Zenith: Verified.

4.3 Medium Risk

A total of 7 medium risk findings were identified.

[M-1] Automatic fee sales can under-mint liquidity additions

SEVERITY: Medium	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Low

Target

- DaimonV2.sol

Description:

`DaimonV2._transfer()` treats every transfer to `uniswapV2Pair` as a sale. This includes the token transfer performed by PancakeSwap's `addLiquidityETH()`. If the Daimon contract holds at least `minimumTokensBeforeSwap`, `_transfer()` calls `_swapAccumulatedFees()` before crediting the liquidity provider's tokens to the pair.

At that point, the router has already read the pair reserves, selected the token and BNB contribution amounts, and performed the applicable checks governed by `amountTokenMin` and `amountETHMin`. Selling the accumulated DMN increases the pair's DMN balance and reserve while decreasing its WBNB balance and reserve. Any subsequent buyback also changes the same reserves. The contribution amounts calculated by the router are therefore stale when the outer liquidity addition resumes.

The full sequence is:

- `addLiquidityETH()` reads the current reserves, calculates the proportional contribution, and checks the applicable minimum amounts.
- The router transfers DMN to the pair, which enters `DaimonV2._transfer()` and triggers `_swapAccumulatedFees()`.
- The nested sale adds DMN to the pair, removes WBNB, and updates the pair reserves before the liquidity provider's transfer is applied.
- The outer transfer completes and the router deposits the WBNB amount calculated from the earlier reserve ratio without reading the reserves again.
- The pair's `mint()` calculates LP shares from the smaller of the two proportional balance increases. After a net DMN sale, the DMN side limits the minted shares and part of the WBNB contribution mints no LP shares.

- `mint()` then updates the reserves to the pair's complete balances, including the entire WBNB deposit. The unmatched WBNB becomes pool backing shared with incumbent LP holders.

The configured router minimums do not protect the liquidity provider. They constrain the contribution amounts selected from the reserve snapshot before the token callback. They do not enforce the reserve ratio after the transfers or a minimum amount of LP tokens, so the nested reserve change does not cause the liquidity addition to revert.

This condition can occur accidentally when organically accumulated fees reach the threshold and the next transfer to the pair is a liquidity addition. It can also be induced intentionally, although this is less likely because profitable exploitation requires a material incumbent LP position, a sufficiently large pending liquidity addition, and transaction ordering. Such a party can top up the Daimon contract to the swap threshold immediately before the liquidity addition, then withdraw its incumbent LP position after the under-minted deposit has increased the backing of its shares.

Recommendations:

Do not execute fee sales or buybacks synchronously from transfers to the AMM pair. Move fee processing to a separate permissionless operation so it completes before the router reads reserves. If the existing transfer behavior must remain, use a liquidity integration that revalidates the post-callback reserve ratio and enforces a minimum LP token output.

Daimon DAO: Resolved with `@043d8b6...` (PR 25) — automation is skipped when msg.sender is the configured router. Third leg of the plan, alongside #27's isolation and #28's per-block budgets.

One consequence worth flagging, since it changes a property we stated in scoping. Ordinary sells also reach _transfer through the router, so they no longer trigger fee conversion either. Conversion now depends on a direct transfer to the pair — permissionless, one wei is enough, bounded by the #28 budgets, but a liveness dependency the protocol didn't have before.

We verified against the canonical periphery before accepting this rather than assuming it. Sells were never exposed to the staleness in #1: in the fee-supporting variant the output is computed after the token callback and amountOutMin is checked against what actually arrives. Liquidity additions differ in kind — their minimums are consumed before the callback and mint() has no minimum output at all, which is the whole finding.

But the callee cannot distinguish them: at the token leg both are byte-identical transferFrom(holder → pair) with msg.sender == router, and the EVM gives no introspection into the caller's frame. The obvious heuristic — probing for a WBNB surplus on the pair — fails because addLiquidityETH transfers the token before depositing WETH, so the surplus is zero exactly when it would matter.

On the sell side the skip is an improvement, not a cost: sellers no longer pay the automation's gas inside their own transaction, nor have their slippage budget consumed by a swap

guaranteed to land immediately before theirs.

Documented in THREAT_MODEL §8 with the reasoning, and in the mainnet checklist as an operational item.

Zenith: Verified. The change correctly prevents fee swaps and buybacks during liquidity additions through the configured PancakeSwap V2 router. Third-party routers or custom liquidity integrations that transfer DMN directly to the pair remain exposed to the original stale-reserve behavior and should be treated as unsupported. Router-mediated sells also no longer trigger fee processing, so conversion cadence now depends on permissionless direct-to-pair transfers, as indicated by the Daimon team.

[M-2] Uncaught automation failures can block sells and liquidity bootstrap

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- src/DaimonV2.sol#L355-L385
- src/DaimonV2.sol#L459-L545
- src/DaimonV2.sol#L679

Description:

DaimonV2 executes fee sales, buybacks, and downstream payouts synchronously inside `_transfer()` before applying the triggering user transfer.

```
if (
    !_inSwap &&
    swapAndLiquifyEnabled &&
    to == uniswapV2Pair &&
    overMin
) {
    _swapAccumulatedFees(
        minimumTokensBeforeSwap
    );
}

if (
    !_inSwap &&
    buyBackEnabled &&
    to == uniswapV2Pair
) {
    // ...
    _buyBackAndBurn(spendAmount / 20);
}
```

Only the final router swaps are wrapped in `try/catch`. Both automation paths query the router before entering the protected call:

```
uint256[] memory quote =
    uniswapV2Router.getAmountsOut(
        tokenAmount,
        path
    );

try uniswapV2Router
    .swapExactTokensForETHSupportingFeeOnTransferTokens(
        tokenAmount,
        minOut,
        path,
        address(this),
        block.timestamp
    )
{} catch {}
```

A swap failure is caught, but a preceding quote failure propagates through `_transfer()` and reverts the triggering transfer. This can occur when either pair reserve is zero, the router reverts, or quote return data is invalid.

After a successful fee sale, DaimonV2 also pushes BNB synchronously to the marketing wallet and Staking contract:

```
(bool ok1, ) =
    marketingWallet.call{
        value: toMarketingWallet
    }("");

require(
    ok1,
    "DaimonV2: marketing transfer failed"
);

IDaimonStakingNotifier(stakingContract)
    .notifyRewardAmount{
        value: toStaking
    }(toStaking);
```

A rejecting marketing wallet or reverting Staking contract rolls back the internal swap and triggering user transfer. The contract's inventory is restored by the revert, so subsequent pair-bound transfers repeatedly encounter the same failure.

Affected transfers include ordinary sells, direct transfers into the pair, initial liquidity additions, and liquidity reseeding. Wallet-to-wallet transfers are not affected because automation runs only when `to == uniswapV2Pair`.

The pair is created with zero reserves while both automation paths are enabled. DaimonV2 also accepts arbitrary BNB:

```
receive() external payable {}
```

An attacker can donate slightly more than one BNB before initial liquidity. The next token transfer used to seed the pair activates the buyback branch. `_buyBackAndBurn()` calls `getAmountsOut()` against the empty pair, which reverts before the caught swap is reached, blocking initial liquidity.

An attacker can similarly donate at least `minimumTokensBeforeSwap` DMN to activate the fee-sale quote branch. Rejecting direct BNB transfers is not a sufficient defense because native currency can potentially be forced into a contract outside its normal receive path.

No funds are lost in the reverted transaction. The impact is persistent sell-side and liquidity liveness failure until governance disables or repairs the affected automation branch.

Recommendations:

It is recommended to make every automation branch fully fail-open.

Before quoting, read the pair reserves and skip automation when either reserve is zero:

```
(
    uint112 reserve0,
    uint112 reserve1,

) = IUniswapV2Pair(uniswapV2Pair)
    .getReserves();

if (reserve0 == 0 || reserve1 == 0) {
    emit AutomationSkipped("zero reserves");
    return;
}
```

Wrap `getAmountsOut()` itself and validate the returned array and output before indexing or calculating `minOut`.

```
try uniswapV2Router.getAmountsOut(
    tokenAmount,
    path
) returns (uint256[] memory amounts) {
    if (
        amounts.length < 2 ||
        amounts[1] == 0
```

```
    ) {
        emit AutomationSkipped(
            "invalid quote"
        );
        return;
    }

    // Attempt the swap.
} catch {
    emit AutomationSkipped("quote failed");
    return;
}
```

For complete isolation, execute fee sales and buybacks through separate restricted external self-calls and catch each entire subcall from `_transfer()`. The wrappers must be callable only by `address(this)` and preserve existing swap and reentrancy guards. This ensures quote handling, accounting, router interaction, and recipient behavior cannot revert the user transfer.

Replace synchronous recipient pushes with explicit accounting:

- record marketing, Staking, and buyback BNB separately;
- let marketing withdraw its pending allocation;
- provide a retryable Staking notification function;
- prevent buybacks from spending failed recipient allocations; and
- emit events for failed and later completed payouts.

Default automation to disabled until both pair reserves and all recipients have been verified, or use an atomic/staged liquidity bootstrap that skips automation while reserves are zero.

Add PancakeSwap-compatible integration tests covering empty and drained reserves, initial liquidity, liquidity reseeding, quote failures, invalid quote data, rejecting recipients, arbitrary BNB and DMN donations, forced native balances, and confirmation that every automation failure leaves the triggering transfer executable.

Daimon DAO: Resolved with `@e5cb1f4009...` and `@96e7563513...` The fee-sale and buyback quote paths now fail open when pool reserves are empty or router quotes revert or return invalid data, closing both the DMN- and BNB-donation variants that could block pair-bound transfers and liquidity bootstrap.

Zenith: Verified.

[M-3] Dust transfers bypass aggregate automation pacing

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- src/DaimonV2.sol#L355-L385
- src/DaimonV2.sol#L459-L485
- src/DaimonV2.sol#L511-L545

Description:

DaimonV2 allows transfers into `uniswapV2Pair` to trigger automated fee-token sales and BNB-funded buybacks. The limits applied to these operations constrain only one call; no aggregate per-block or per-epoch pacing exists.

Automation executes before the triggering transfer is applied:

```
if (
    !_inSwap &&
    swapAndLiquifyEnabled &&
    to == uniswapV2Pair &&
    overMin
) {
    _swapAccumulatedFees(
        minimumTokensBeforeSwap
    );
}

if (
    !_inSwap &&
    buyBackEnabled &&
    to == uniswapV2Pair
) {
    uint256 ethBalance = address(this).balance;
    if (
        ethBalance > 1 ether &&
        _tTotal > MIN_SUPPLY
    ) {
        uint256 spendAmount =
```

```
            ethBalance > buyBackUpperLimit
                ? buyBackUpperLimit
                : ethBalance;

        _buyBackAndBurn(spendAmount / 20);
    }
}
```

The triggering transfer needs only a positive amount. At one token wei, all percentage fees round to zero:

```
uint256 tFee =
    (1 * taxFee) / 1000; // 0

uint256 tLiquidity =
    (1 * liquidityFee) / 1000; // 0
```

An attacker contract can repeatedly call:

```
token.transfer(uniswapV2Pair, 1);
```

Each iteration costs one DMN wei plus gas. `lockSwap` and `nonReentrant` prevent nested execution during an individual internal operation, but they reset afterward and do not restrict sequential calls.

When fee-sale automation is enabled and sufficient inventory exists, each successful iteration sells another `minimumTokensBeforeSwap` chunk. Repetition can process every full chunk in one transaction or block, leaving only a residual balance below the threshold.

When buybacks are enabled, BNB exceeds one ether, and supply remains above `MIN_SUPPLY`, each successful iteration executes another buyback slice. Repetition can continue until the pre-call BNB balance no longer exceeds one ether; the final buyback may leave slightly less than one ether.

Consequently, `minimumTokensBeforeSwap`, `buyBackUpperLimit`, and the 5% per-call buyback fraction do not limit aggregate execution. An unprivileged dust-funded actor can choose when the protocol realizes multiple fee-token chunks and spends most eligible buyback inventory.

The automated trades use quotes derived from the pair's current reserves. An attacker can combine reserve manipulation with repeated triggers to expose multiple protocol orders to adversarial pricing before reversing the manipulation. Fee sales and buybacks trade in opposite directions, so the attacker must target the branch with material inventory, exploit net order flow, or adjust reserves between iterations.

The sequence requires enabled automation, available inventory, a functioning pair, and successful external operations. Failed swaps do not consume the corresponding token or BNB

inventory.

Recommendations:

It is recommended to enforce caller-independent aggregate limits for automated token sales and BNB spending.

Track separate per-block or short-epoch budgets for:

- cumulative DMN sold;
- cumulative BNB spent;
- fee-sale attempts; and
- buyback attempts.

```
if (automationBlock != block.number) {
    automationBlock = block.number;
    tokensSoldThisBlock = 0;
    bnbSpentThisBlock = 0;
    feeSwapAttemptedThisBlock = false;
    buybackAttemptedThisBlock = false;
}
```

Consume a short-lived attempt allowance before external interaction so a caller cannot retry caught failures indefinitely in the same block. Refactor the automation functions to report success and actual execution amounts, then count only successful sales and expenditure against longer-lived aggregate budgets.

Keep fee-sale and buyback budgets independent. Enforce governance-bounded maximum values so governance cannot accidentally configure the limits as ineffective.

A meaningful minimum triggering amount and separate cooldowns are useful defense in depth but cannot replace aggregate limits because larger triggering transfers remain repeatable.

Use an independent TWAP or robust oracle with staleness and deviation checks for automated trade validation. Aggregate pacing limits exposure but do not make same-block reserve quotes manipulation-resistant.

Add PancakeSwap-compatible integration tests covering repeated dust triggers, multiple inventory chunks, fee sales, buybacks, caught failures, reserve manipulation, and budget resets.

Daimon DAO: Resolved with `@f42a629e22 ...`

Zenith: Verified.

[M-4] Reflections accrued by the AMM pair can be permissionlessly skimmed

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- src/DaimonV2.sol#L244-L257
- src/DaimonV2.sol#L327-L349
- src/DaimonV2.sol#L388-L435

Description:

DaimonV2 excludes only `deadAddress` from reflection accounting. The PancakeSwap pair is created during initialization but remains reflection-included.

```
_isExcludedFromReward[deadAddress] = true;
_excluded.push(deadAddress);

uniswapV2Pair =
    IUniswapV2Factory(uniswapV2Router.factory())
        .createPair(
            address(this),
            uniswapV2Router.WETH()
        );
```

Each reflection fee reduces `_rTotal`, lowering the reflected-unit conversion rate and increasing the token-denominated balance of every reflection-included holder.

```
function _reflectFee(
    uint256 rFee,
    uint256 tFee
)
    private
{
    _rTotal -= rFee;
    _tFeeTotal += tFee;
}
```

When an unrelated taxed transfer occurs between pair reserve updates, the pair's DMN balance increases passively while its recorded reserve remains unchanged:

```
balanceOf(pair) > recordedReserve
```

PancakeSwap V2 exposes permissionless `skim(to)`, which transfers this balance-over-reserve excess to an arbitrary recipient. An unprivileged bot can therefore extract the reflected excess before a swap, `sync()`, or liquidity operation incorporates it into the pool.

The transfer from the pair is taxed, so the caller receives the excess net of fees. It is also subject to `maxTxAmount`; if the entire excess exceeds that limit, `skim()` reverts because it cannot request a partial amount. This is only a timing constraint because bots can monitor and harvest smaller profitable excesses before the limit is reached. The skim transfer's own reflection can leave a smaller new excess, but this does not create an amplification loop that drains recorded reserves.

The issue does not directly remove recorded AMM reserves. It diverts passive reflection value held by the LP-backed pair that could otherwise benefit liquidity providers through synchronization or balance-based liquidity removal.

The extractable value depends on the reflection fee, organic taxed transfer volume, the pair's share of reflection-included supply, reserve-update timing, transfer fees, and gas costs. An attacker generally cannot profitably create unlimited excess alone because generating reflections costs the full transfer fee while the pair receives only its proportional reflection share.

Recommendations:

It is recommended to add the pair to the fixed reflection-excluded set immediately after pair creation and before adding liquidity.

```
uniswapV2Pair =
    IUniswapV2Factory(uniswapV2Router.factory())
        .createPair(
            address(this),
            uniswapV2Router.WETH()
        );

_isExcludedFromReward[uniswapV2Pair] = true;
_excluded.push(uniswapV2Pair);
```

Because a newly created pair has a zero token balance, no reflected-balance conversion is required.

For an already funded pair, implement a one-time UUPS upgrade migration that atomically:

1. reads `balanceOf(pair)` while the pair remains reflection-included;
2. writes that balance to `_tOwned[pair]`;
3. marks the pair as reflection-excluded;
4. registers it in `_excluded` exactly once; and
5. reconciles existing balance-over-reserve excess through an explicit policy, such as an atomic `sync()` after exclusion.

```
uint256 pairBalance = balanceOf(pair);

_tOwned[pair] = pairBalance;
_isExcludedFromReward[pair] = true;
_excluded.push(pair);
```

The existing excess must be handled separately because exclusion prevents future accrual but does not itself update AMM reserves.

Do not introduce a general runtime reflection-exclusion setter. A fixed initialization or one-time migration preserves the project's intended immutable exclusion model and avoids mutable RFI bookkeeping risks.

If the pair must remain reflection-included, redesign reflection allocation or explicitly document and accept that pair reflection is public MEV. Periodic `sync()` alone is not reliable because it can be front-run by `skim()` and changes pool reserves and price.

Add PancakeSwap-compatible integration tests covering passive reflection accrual, balance/reserve drift, `skim()`, transfer fees, `maxTxAmount`, and post-migration reserve reconciliation.

Daimon DAO: Resolved with `@ed656e7 ...` — the pair is added to the fixed reflection exclusion set immediately after createPair() in initialize().

Zenith: Verified.

[M-5] Reflection accrued by Staking is unrecoverable through current interfaces

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- src/DaimonV2.sol#L246-L254
- src/DaimonV2.sol#L327-L435
- src/DaimonStaking.sol#L82-L86
- src/DaimonStaking.sol#L178-L203

Description:

Only `deadAddress` is excluded from DaimonV2 reflections. Configuring the Staking contract excludes it from transfer fees, but fee exclusion and reflection exclusion are independent. Staking remains an ordinary reflection-receiving token holder.

As taxed transfers reduce `_rTotal`, the reflection conversion rate changes and `balanceOf(address(staking))` grows passively while Staking holds user principal.

```
function balanceOf(address account)
    public
    view
    returns (uint256)
{
    if (_isExcludedFromReward[account]) {
        return _tOwned[account];
    }

    return _tokenFromReflection(_rOwned[account]);
}
```

Staking separately records deposited principal in `totalStakedAmount`. Withdrawals return exactly the amount recorded in each lock rather than a proportional share of the contract's current token balance.

```
uint256 amount = lockData.amount;
```

```
lockData.withdrawn = true;
totalStaked[msg.sender] -= amount;
totalStakedAmount -= amount;

bool ok = daimonToken.transfer(
    msg.sender,
    amount
);
```

Consequently, reflected DMN remains in Staking after principal withdrawals. The excess balance is:

```
max(
    token.balanceOf(staking) - totalStakedAmount,
    0
)
```

This excess can also include accidental direct token transfers, so it should be treated as general surplus rather than exclusively as reflection.

The contract remains overcollateralized, and the issue does not threaten withdrawal of recorded principal. Users are promised exact principal plus BNB rewards, not explicit DMN reflection rewards. The impact is therefore undefined and inaccessible protocol value rather than theft of principal or clearly assigned user yield.

`DaimonStaking` has no function capable of transferring the surplus and is not upgradeable. After every user withdraws, the excess remains held by the contract with no recovery path through the current interfaces. It can become economically material as transaction volume and Staking's share of circulating supply increase, although it remains bounded by token supply and total reflection activity.

DaimonV2 is UUPS-upgradeable, so governance could theoretically deploy extraordinary token logic that directly changes token accounting to recover the balance. This is a sensitive, non-standard recovery path and does not make the surplus accessible through ordinary protocol operations.

The repository deliberately permits only `deadAddress` to be excluded from reflection. If surplus accumulated by Staking is intended as a permanent tokenomic sink, that ownership policy and its expected magnitude should be documented explicitly.

Recommendations:

It is recommended to define ownership of Staking surplus explicitly and add a Timelock-governed, principal-safe transfer function before deployment.

The destination should follow a documented policy, such as a predetermined DAO treasury, voted burn path, or separately audited staker-reward distributor. The function must never

reduce Staking's balance below `totalStakedAmount`.

```
error InvalidSurplusRecipient();

event SurplusTransferred(
    address indexed recipient,
    uint256 amountDebited,
    uint256 amountReceived
);

function transferSurplus(
    address recipient,
    uint256 requestedAmount
)
    external
    onlyGovernance
    nonReentrant
{
    if (
        recipient == address(0) ||
        recipient == address(this)
    ) revert InvalidSurplusRecipient();

    uint256 balanceBefore =
        daimonToken.balanceOf(address(this));

    if (balanceBefore <= totalStakedAmount) return;

    uint256 available =
        balanceBefore - totalStakedAmount;

    uint256 amount =
        requestedAmount < available
            ? requestedAmount
            : available;

    if (amount == 0) return;

    uint256 recipientBefore =
        daimonToken.balanceOf(recipient);

    bool ok = daimonToken.transfer(
        recipient,
        amount
    );
    require(ok, "DaimonStaking: transfer failed");
```

```
    uint256 balanceAfter =
        daimonToken.balanceOf(address(this));

    require(
        balanceAfter >= totalStakedAmount,
        "DaimonStaking: principal impaired"
    );

    uint256 recipientAfter =
        daimonToken.balanceOf(recipient);

    emit SurplusTransferred(
        recipient,
        balanceBefore - balanceAfter,
        recipientAfter - recipientBefore
    );
}
```

Keep Staking excluded from transfer fees or retain the balance-delta accounting above so governance and monitoring can distinguish the amount debited from the amount received.

Add tests confirming that:

- reflection increases Staking's balance above recorded principal;
- accidental token transfers are classified as surplus;
- surplus transfers cannot reduce the balance below `totalStakedAmount`;
- all principal remains withdrawable afterward;
- zero-surplus and zero-amount calls are harmless; and
- fee configuration changes cannot bypass the principal reserve.

`DaimonStaking` is not upgradeable. Existing instances cannot add this function in place, and replacing Staking alone leaves the old surplus behind. Recovery of already accrued surplus would require an extraordinary DaimonV2 upgrade or acceptance of the balance as inaccessible. The correction should be applied before mainnet deployment.

Daimon DAO: Resolved with `@c98bc801ed ...`

Zenith: Verified.

[M-6] A minimal first stake can atomically capture the zero-staker reward backlog

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- src/DaimonStaking.sol#L132-L175
- src/DaimonStaking.sol#L246-L294

Description:

When `totalVotingPower` is zero, incoming BNB rewards are stored in `undistributedRewards`. The next notification made while any voting power exists combines the entire backlog with the new notification and distributes it immediately among the current stakers.

```
if (totalVotingPower == 0) {
    undistributedRewards += amount;
    emit RewardNotified(amount);
    return;
}

uint256 toDistribute = amount + undistributedRewards;
undistributedRewards = 0;
rewardPerVotingPowerStored +=
    (toDistribute * REWARD_PRECISION) / totalVotingPower;
```

Staking requires only a nonzero amount. Under the default 1× lock option, one token wei grants one voting-power wei.

```
if (amount == 0) revert ZeroAmount();

uint256 vp =
    (amount * opt.multiplierX1000) / 1000;
```

`notifyRewardAmount()` is permissionless and accepts a zero-value notification because it only checks that `msg.value == amount`. An attacker can therefore:

1. Wait for nonzero `undistributedRewards` while `totalVotingPower` is zero.
2. Stake one token wei using the 1× option.
3. Call `notifyRewardAmount(0)`.
4. Claim the entire queued reward backlog.
5. Withdraw the negligible principal after the 30-day lock.

For a backlog of `B` wei and one voting-power wei, the accumulator credits exactly `B` wei to the attacker. A contract can perform the stake, zero-value notification, and claim sequentially within one transaction, eliminating the race between becoming the first staker and triggering distribution.

The zero-value notification is only an immediate trigger. Rejecting it does not fix the underlying issue because the next legitimate notification would distribute the same backlog to the attacker. Requiring a positive notification is similarly ineffective because the attacker can contribute one wei of BNB.

The attack applies before the first stake and during any later interval in which every position has been withdrawn. It captures only `undistributedRewards`; rewards already attributed to previous stakers, direct unaccounted BNB, and rounding dust are unaffected.

Source comments, tests, and the threat model acknowledge that zero-staker rewards are queued and later distributed at the first useful notification. They do not disclose that an ordinary holder can atomically capture the entire backlog with one token wei. If this economic allocation is intentionally accepted, the negligible-cost capture consequence should be documented explicitly.

Recommendations:

It is recommended to define an explicit ownership policy for rewards received while no eligible stakers exist. Do not automatically merge the zero-staker backlog into the first subsequent notification.

The clearest approach is to account for those funds in a separate reserve and route them to a predetermined treasury or release them only through an explicit governance action.

```
if (totalVotingPower == 0) {
    zeroStakerReserve += amount;
    emit ZeroStakerRewardReserved(amount);
    return;
}

// Historical zero-staker reserves are not included.
_distribute(amount);
```

If future stakers are intended to receive the backlog, distribute it over a fixed epoch using snapshot-based eligibility, stake age, and a meaningful participation threshold. Streaming alone is insufficient if the attacker remains the only staker, while minimum stake amounts and positive-notification requirements are only economic deterrents.

Add tests confirming that:

- a one-wei first stake cannot claim historical rewards;
- zero-value and subsequent legitimate notifications cannot release the entire reserve to a newly created position;
- ordinary rewards remain proportional among eligible stakers; and
- reserve accounting always matches the BNB retained for its configured destination.

`DaimonStaking` is not upgradeable. The reward policy should be corrected before mainnet deployment. Remediating an existing deployment requires coordinated handling of its reward backlog and active locks and may require replacing both Staking and the Governor that references it immutably.

Daimon DAO: Resolved with `@c98bc801ed ...`

Zenith: Verified.

[M-7] Guardian can indefinitely censor governance and preserve a global pause

SEVERITY: Medium	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Low

Target

- script/Deploy.s.sol#L92-L113
- src/DaimonV2.sol#L139-L175
- src/DaimonV2.sol#L651-L663
- src/DaimonGovernor.sol#L210-L224
- src/DaimonTimelock.sol#L54-L69
- src/DaimonTimelock.sol#L113-L118

Description:

The deployment initializes the same guardian address with three independently managed authorities:

- token `GUARDIAN_ROLE`;
- Governor cancellation authority; and
- Timelock `CANCELLER_ROLE`.

Only the ability to activate a token pause expires. `guardianExpiry` prevents setting `paused` to `true` after 36 months, but it does not clear a pause that was activated earlier.

```
function setPaused(bool _paused) external onlyRole(GUARDIAN_ROLE) {
    if (_paused && block.timestamp >= guardianExpiry) {
        revert GuardianExpired();
    }
    paused = _paused;
}
```

Token transfers check only the persistent boolean, without considering `guardianExpiry` or an automatic pause duration.

```
modifier whenNotPaused() {
    if (paused) revert ContractIsPaused();
```

```
    _;
}
```

The Governor guardian can cancel any unexecuted proposal without a time limit. Replacing the Governor guardian requires an executable proposal targeting `setGuardian()`, which the current guardian can itself cancel.

```
function cancel(uint256 id) external onlyGuardian {
    Proposal storage p = proposals[id];
    if (p.executed) revert AlreadyExecuted();
    p.canceled = true;
}
```

The deployment-time guardian is also granted Timelock `CANCELLER_ROLE`. That role has no automatic expiry, and its holder can cancel scheduled operations intended to revoke the role, rotate the token guardian, upgrade the token, or perform unrelated governance actions.

```
function cancel(bytes32 id) external onlyRole(CANCELLER_ROLE) {
    Operation storage op = operations[id];
    require(!op.executed, "DaimonTimelock: already executed");
    op.canceled = true;
}
```

After bootstrap permissions are renounced, governance is the only mechanism capable of changing these authorities. A compromised guardian that remains active can continue submitting cancellation transactions during the voting and seven-day Timelock windows, indefinitely preventing recovery proposals from executing. There is no independent administrator or non-cancellable removal path.

The guardian can additionally activate a pause immediately before expiry and refuse to unpause. Passing `guardianExpiry` does not make that pause ineffective. Governance could grant `GUARDIAN_ROLE` to a recovery address or upgrade the token, but the compromised guardian can cancel the required proposal or Timelock operation.

This blocks all token transfers, including trading, liquidity operations, staking deposits, staking withdrawals, and migration payouts. Stakers cannot recover their principal while the pause remains effective.

If a pause covers the immutable migration deadline, claims revert atomically, so users retain their old tokens. However, after the deadline they permanently lose access to this migration mechanism. Unclaimed supply can only be swept later if the token is eventually unpaused and governance is no longer censored.

The intended production guardian is a multisig, reducing the likelihood of compromise but not providing an on-chain recovery mechanism. The behavior also contradicts the docu-

mented assumption that guardian authority and censorship end after 36 months.

Recommendations:

It is recommended to establish one fixed, non-extendable guardian expiration time and enforce it consistently in the token, Governor, and Timelock. After that time, both cancellation functions should reject guardian actions even if the corresponding address or role has not yet been removed.

Replace the persistent pause with an automatically expiring pause deadline. Preserve the existing `paused` storage slot when upgrading `DaimonV2`, and append any new state variable to avoid corrupting the UUPS proxy's storage layout.

```
bool public paused;                // retain existing storage slot
uint256 public guardianExpiry;     // existing field
uint256 public pauseUntil;         // append to existing storage

function isPaused() public view returns (bool) {
    return paused && block.timestamp < pauseUntil;
}

modifier whenNotPaused() {
    if (isPaused()) revert ContractIsPaused();
    _;
}

function setPaused(bool shouldPause)
    external
    onlyRole(GUARDIAN_ROLE)
{
    if (!shouldPause) {
        paused = false;
        pauseUntil = 0;
        return;
    }

    if (block.timestamp >= guardianExpiry) revert GuardianExpired();

    uint256 maximumEnd = block.timestamp + MAX_PAUSE_DURATION;
    pauseUntil =
        maximumEnd < guardianExpiry
            ? maximumEnd
            : guardianExpiry;
    paused = true;
}
```

Retain an early-unpause mechanism, but ensure expiration never depends on guardian cooperation.

Provide a recovery path that becomes non-cancellable once guardian authority expires. Preserve users' complete migration opportunity through automatic pause-duration accounting or a guaranteed post-pause claim window; do not rely solely on a governance extension that the guardian could veto.

`DaimonGovernor` and `DaimonTimelock` are not upgradeable. Existing instances cannot apply these controls in place and require replacement with careful role and dependency rewiring. The corrected contracts should be used before mainnet deployment.

Add tests covering a compromised guardian that:

- pauses immediately before expiry and refuses to unpause;
- cancels its own replacement and role revocation;
- attempts Governor and Timelock cancellation after expiry;
- attempts to block an upgrade or unrelated governance action; and
- pauses across the migration deadline.

Daimon DAO: Resolved with `@bd5c042f21 ...`

Zenith: Verified.

4.4 Low Risk

A total of 12 low risk findings were identified.

[L-1] Unbounded migration duration can make claims or final settlement unusable

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- src/DaimonMigration.sol#L76-L87
- src/DaimonMigration.sol#L91-L93
- src/DaimonMigration.sol#L130-L141

Description:

The immutable migration deadline is derived from a constructor parameter without enforcing a positive minimum or operationally meaningful maximum.

```
migrationDeadline =
    block.timestamp + _migrationDurationSeconds;
```

A zero or very short duration leaves only the deployment block, or a similarly impractical interval, available for claims:

```
if (block.timestamp > migrationDeadline) {
    revert MigrationEnded();
}
```

Conversely, a very large but non-overflowing duration can place the deadline beyond any practical operational horizon. This prevents governance from using the only final-settlement path:

```
if (block.timestamp <= migrationDeadline) {
    revert MigrationStillOpen();
}
```

Arithmetic overflow already causes construction to revert under Solidity 0.8.26. The relevant risk is therefore deployment with a technically valid but operationally unusable duration.

The deployment script defaults to 30 days, but `MIGRATION_DURATION` remains externally configurable and is not validated. Because the deadline is immutable, an incorrect production value cannot be corrected after deployment.

This requires deployment misconfiguration rather than an unprivileged attacker, resulting in Low overall severity despite the potentially material lifecycle impact.

Recommendations:

It is recommended to define protocol-approved minimum and maximum migration durations and enforce them in both the constructor and deployment script.

```
error InvalidMigrationDuration();

constructor(
    address _oldDaimon,
    address _newDaimon,
    address _treasury,
    address _governance,
    uint256 _migrationDurationSeconds
) {
    if (
        _migrationDurationSeconds < MIN_MIGRATION_DURATION ||
        _migrationDurationSeconds > MAX_MIGRATION_DURATION
    ) {
        revert InvalidMigrationDuration();
    }

    migrationDeadline =
        block.timestamp + _migrationDurationSeconds;

    // Existing initialization...
}
```

The exact bounds should be selected as protocol policy rather than inferred from the existing 30-day default. Before broadcasting, the deployment script should also print and assert both the supplied duration and resulting absolute deadline.

Daimon DAO: Resolved with `@0d8b09d7a7 ...`

Zenith: Verified.

[L-2] Buyback BNB has no current-interface settlement path at the supply floor

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Medium

Target

- src/DaimonV2.sol#L376-L381
- src/DaimonV2.sol#L459-L469
- src/DaimonV2.sol#L511-L575

Description:

Fee liquidation retains the non-marketing portion of received BNB as buyback inventory. This balance is ordinarily spent only when buybacks are enabled, their effective cap is nonzero, and supply remains above `MIN_SUPPLY`.

```
if (!_inSwap && buyBackEnabled && to == uniswapV2Pair) {
    uint256 ethBalance = address(this).balance;
    if (ethBalance > 1 ether && _tTotal > MIN_SUPPLY) {
        uint256 spendAmount =
            ethBalance > buyBackUpperLimit
                ? buyBackUpperLimit
                : ethBalance;

        _buyBackAndBurn(spendAmount / 20);
    }
}
```

Disabling buybacks or setting an ineffective cap only suspends settlement because governance can restore a usable configuration. The terminal supply-floor case is different:

```
function _buyBackAndBurn(uint256 ethAmount) private {
    if (ethAmount == 0) return;
    if (_tTotal <= MIN_SUPPLY) return;
    // ...
}
```

`burnDeadBalanceToFloor()` can reduce supply to exactly `MIN_SUPPLY`. Once this happens, no ordinary function can spend retained buyback BNB, and the contract exposes no alternative settlement path.

The affected balance is protocol-owned buyback inventory or direct donations, not user principal or an accrued user liability. Governance can also recover it through a future UUPS upgrade, so the funds are inaccessible through the current implementation rather than absolutely unrecoverable.

Recommendations:

It is recommended to define a terminal buyback policy and add a non-reentrant settlement function that becomes callable only once supply reaches `MIN_SUPPLY`.

The recipient should be fixed or tightly constrained during the upgrade that introduces the function. The function should not create a general pre-floor governance withdrawal capability.

```
address payable public terminalBuybackRecipient; // append safely
event TerminalBuybackBalanceSettled(
    address indexed recipient,
    uint256 amount
);

function settleTerminalBuybackBalance()
    external
    onlyRole(GOVERNANCE_ROLE)
    nonReentrant
{
    if (_tTotal != MIN_SUPPLY) revert AboveMinSupply();

    address payable recipient = terminalBuybackRecipient;
    if (recipient == address(0)) revert ZeroAddress();

    uint256 amount = address(this).balance;
    if (amount == 0) return;

    (bool ok,) = recipient.call{value: amount}("");
    require(ok, "DaimonV2: terminal settlement failed");

    emit TerminalBuybackBalanceSettled(recipient, amount);
}
```

Because DaimonV2 is upgradeable, new storage must be appended without altering the existing layout, and the recipient must be initialized through an appropriate upgrade initializer.

Daimon DAO: Resolved with `@a2eebcae6c ...`

Zenith: Verified.

[L-3] Native-BNB liquidity removal charges DMN fees twice without directly bounding final receipt

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Medium

Target

- src/DaimonV2.sol#L209-L257
- src/DaimonV2.sol#L355-L421
- PancakeRouter.sol#L103-L141
- PancakeRouter.sol#L172-L193

Description:

DMN charges fees whenever neither transfer endpoint is fee-exempt:

```
bool takeFee =
    !(isExcludedFromFee[from] || isExcludedFromFee[to]);

uint256 tFee = (tAmount * taxFee) / 1000;
uint256 tLiquidity =
    (tAmount * liquidityFee) / 1000;
uint256 tTransferAmount =
    tAmount - tFee - tLiquidity;
```

The canonical pair and PancakeSwap router are not fee-exempt in the reviewed initialization. PancakeSwap's fee-on-transfer-supporting native-BNB removal consequently performs two taxable DMN transfers:

1. the pair transfers DMN to the router during `burn(address(router))`; and
2. the router forwards its DMN balance to the user.

```
(, amountETH) = removeLiquidity(
    token,
    WETH,
    liquidity,
    amountTokenMin,
```

```
    amountETHMin,
    address(this),
    deadline
);

TransferHelper.safeTransfer(
    token,
    to,
    IERC20(token).balanceOf(address(this))
);
```

Assuming the router held no DMN before the call, both endpoints remain non-exempt, and the execution-time total fee is `f`, gross pair redemption `G` produces approximately:

```
router receipt ≈ G × (1 − f)
user receipt   ≈ G × (1 − f)²
```

The source initializer's 5% fee therefore produces an approximate final receipt of 90.25% of `G`. The actual result depends on the fees configured at execution and is slightly affected by reflection and integer rounding.

`amountTokenMin` is checked inside `removeLiquidity()` against gross amount `G`, before either DMN fee affects the final recipient. It does not directly constrain the router's receipt or the user's final balance increase. A conventional gross slippage value can therefore pass while the user receives materially less DMN.

The ordinary `removeLiquidityETH()` route normally reverts because it attempts to forward gross amount `G` from a router that received less than `G`. It can succeed if fees are zero, an endpoint is fee-exempt, or the router already holds enough DMN to cover the difference.

The deducted value is redistributed through reflection and protocol fee accumulation rather than disappearing. This is an integration and output-protection issue, not an attacker-driven reserve drain.

Recommendations:

It is recommended to support direct DMN/WBNB removal as the preferred flow. Calling `removeLiquidity()` with the user as recipient produces one taxable DMN transfer, after which the user can unwrap WBNB separately.

For either route, calculate a conservative gross `amountTokenMin` from the user's desired final net DMN amount using the execution-time fee configuration and the token's exact integer-rounding rules. For native-BNB removal, this calculation must account for both taxable DMN transfers.

Where an exact final-receipt guarantee is required, use a protocol-specific wrapper that

records the recipient's DMN balance before and after the supporting router call and reverts unless the increase satisfies a user-provided minimum. This protects the final output but does not eliminate the two transfer fees.

Interfaces should disclose whether the user will receive WBNB or native BNB, the number of taxable DMN transfers, the expected final DMN receipt, and the fee assumptions used in the quote.

The canonical router should not be blanket fee-exempt. Because the supporting function forwards its entire DMN balance, such an exemption could enable unintended fee-free transfers through pre-depositing DMN and invoking liquidity removal.

Daimon DAO: Resolved with `@41b4d891e6 ...` by explicitly documenting the accepted double-fee behavior and requiring the interface to disclose both taxable transfers and the expected final net DMN receipt before signing.

Zenith: Verified.

[L-4] Standard liquidity additions use gross DMN amounts despite taxed pair receipts

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Medium

Target

- src/DaimonV2.sol#L244-L257
- src/DaimonV2.sol#L384-L421

Description:

The canonical pair is not fee-exempt by default. Consequently, a DMN transfer from a non-exempt liquidity provider to the pair is taxed whenever the configured fees are nonzero:

```
bool takeFee =
    !(isExcludedFromFee[from] || isExcludedFromFee[to]);

uint256 tFee = (tAmount * taxFee) / 1000;
uint256 tLiquidity = (tAmount * liquidityFee) / 1000;
uint256 tTransferAmount =
    tAmount - tFee - tLiquidity;
```

PancakeSwap V2 does not provide a fee-on-transfer-specific liquidity-addition function. Its standard functions first select and validate gross token amounts using the stored reserves, then transfer those amounts and ask the pair to mint LP tokens:

```
(amountToken, amountETH) = _addLiquidity(...);
TransferHelper.safeTransferFrom(
    token,
    msg.sender,
    pair,
    amountToken
);
liquidity = IPancakePair(pair).mint(to);
```

The pair calculates liquidity from its actual balances relative to its stored reserves. Its usable DMN increase can include the post-fee transfer, reflection generated by the transfer, and

any pre-existing DMN balance surplus accumulated since the reserves were last updated. Therefore, the precise result is state-dependent.

When the pair's usable DMN increase is less than the gross DMN amount selected by the router, the DMN side limits LP issuance while the router still supplies BNB/WBNB according to the gross reserve ratio. The provider can consequently receive fewer LP tokens than the supplied values suggest, with disproportionate BNB/WBNB remaining in the pool. This can transfer value to existing LP holders and create an arbitrageable reserve-ratio change. A sufficient pre-existing DMN surplus may partially or completely offset this effect.

`amountTokenMin` does not protect against this outcome because it validates the gross DMN amount selected before transfer. It does not constrain the pair's final DMN increase or impose a minimum number of LP tokens to be minted.

For an empty pool, there are no existing LP holders to dilute. However, the initial price is established from the DMN actually received by the pair and the supplied BNB/WBNB. Calculating the BNB/WBNB contribution from the provider's gross DMN input can therefore initialize the pool at an unintended price.

The issue is inactive when transfer fees are zero or either transfer endpoint is fee-exempt. Exempting the router alone has no effect because it is the spender rather than the `from` or `to` address.

Recommendations:

It is recommended to provide an atomic liquidity adapter that supports taxed pair transfers. For an established pool, it should:

1. establish a balance baseline after accounting for any pre-existing balance/reserve surplus;
2. transfer DMN to the pair;
3. read the pair's updated reserves and actual usable DMN increase;
4. calculate and supply the proportional BNB/WBNB amount;
5. mint LP tokens directly through the pair; and
6. enforce user-specified minimum net DMN receipt and minimum LP output.

The standard router should not be called after measuring the DMN receipt because it would perform another DMN transfer and reproduce the mismatch.

If tax-free liquidity additions are intended, a restricted fee-exempt adapter may instead be used. It must only add liquidity to the canonical pair and send the resulting LP tokens to the depositor so that it cannot become a general-purpose fee-bypass route.

For initial liquidity, calculate the BNB/WBNB contribution from the pair's actual DMN receipt and verify the resulting reserve ratio. Interfaces should also disclose the expected fee, pair

receipt, LP output, and price impact. Disclosure alone does not remediate established-pool additions through the standard router.

The pair itself should not be blanket-exempted unless disabling transaction fees on ordinary buys and sells is an intended tokenomics change.

Daimon DAO: Resolved with `@41b4d891e6 ...` by explicitly documenting the accepted taxed-liquidity behavior, requiring the interface to disclose the expected fee, actual pair receipt and LP output before signing, and recording the actual-receipt and reserve-ratio checks required for initial liquidity.

Zenith: Verified.

[L-5] An unbounded proposal threshold can permanently disable proposal creation

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- src/DaimonGovernor.sol#L111-L138
- src/DaimonGovernor.sol#L235-L239

Description:

The Governor accepts an unrestricted proposal threshold during deployment and through its governance setter:

```
proposalThreshold = _proposalThreshold;

function setProposalThreshold(uint256 threshold) external {
    require(msg.sender == address(timelock), "DaimonGovernor: only via
    timelock");
    proposalThreshold = threshold;
    emit ProposalThresholdSet(threshold);
}
```

Creating a proposal requires one account to hold at least the configured voting power:

```
if (staking.votingPower(msg.sender) < proposalThreshold) {
    revert InsufficientVotingPower();
}
```

Governance can set the threshold above the voting power any account can attain. The threshold is denominated in derived voting-power units, not DMN. Under the reviewed limits, even an account controlling the entire maximum supply of one thousand billion DMN and staking it at the maximum 10× multiplier could attain at most ten thousand billion voting-power units, while the setter accepts values up to `type(uint256).max`.

Under the intended post-deployment role configuration, only the Governor can schedule Timelock operations. If an unattainable threshold is executed, the Governor cannot create the proposal needed to lower it. Recovery is possible only if an appropriate proposal or

Timelock operation already exists, or alternative Timelock proposer and executor permissions were previously configured.

The constructor has the same missing validation. The provided deployment script uses a safe fixed threshold of 1,000 DMN, so this deployment route requires a modified or manual deployment.

The issue requires an erroneous deployment or a governance-approved configuration change. It does not provide an unprivileged attack path, and the voting and Timelock periods provide an opportunity to cancel an unsafe proposal. However, once executed without a recovery path, it can permanently prevent upgrades, parameter changes, role rotations, and governance-based incident response.

Recommendations:

It is recommended to define a fixed maximum proposal threshold that keeps proposal creation realistically attainable under the intended token distribution. Enforce the same maximum in both the constructor and `setProposalThreshold()`.

The maximum should reflect the protocol's governance-accessibility policy rather than merely the theoretical maximum voting power. Add tests confirming that values above the bound revert in both deployment and governance-update flows.

A nonzero minimum is necessary only if permissionless proposal creation is not intended. If irreversible governance ossification is desired, implement it as an explicit, documented action instead of relying on an extreme threshold value.

Daimon DAO: Resolved with `@2c247b9 ...` — MAX_PROPOSAL_THRESHOLD enforced in both the constructor and the setter.

The value is derived from the immutable supply floor: MIN_SUPPLY / 100 = 210M voting power units. At today's supply that's 0.005%; in the terminal scenario at the floor it's 1% at 1x lock — still reachable by a coalition, never a gate. The Governor doesn't import the token, so the derivation is a comment plus a test asserting the constant still equals token.MIN_SUPPLY() / 100.

Zenith: Verified.

[L-6] Unbounded Timelock delay can permanently disable new governance scheduling

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- src/DaimonTimelock.sol#L54-L56
- src/DaimonTimelock.sol#L79-L90
- src/DaimonTimelock.sol#L120-L125

Description:

Governance can change `minDelay` through an authorized Timelock self-call. The setter enforces the seven-day minimum but no maximum:

```
function updateDelay(uint256 newDelay) external {
    require(
        msg.sender == address(this),
        "DaimonTimelock: only via self-call (governance proposal)"
    );
    require(
        newDelay >= MIN_DELAY,
        "DaimonTimelock: below MIN_DELAY"
    );

    minDelay = newDelay;
}
```

New operations must use a delay of at least `minDelay`. Scheduling then calculates the readiness timestamp using checked arithmetic:

```
if (delay < minDelay || delay < MIN_DELAY) {
    revert DelayTooShort();
}

uint256 readyTimestamp = block.timestamp + delay;
```

Governance can set:

```
minDelay > type(uint256).max − block.timestamp
```

Afterward, every permitted `delay ⩾ minDelay` causes the readiness calculation to overflow and revert. Lowering `minDelay` requires another scheduled Timelock self-call, but the repair operation can no longer be scheduled.

Previously scheduled operations remain executable. Recovery is possible if a suitable operation capable of lowering the delay was already scheduled before the unsafe update. Otherwise, new governance scheduling becomes permanently unavailable.

Even values below the overflow threshold can make governance economically unusable if the resulting delay is impractically long.

This requires an explicitly approved governance action and cannot be triggered by an unprivileged caller. Ordinary protocol activity and previously scheduled operations can continue, but new upgrades, role changes, parameter updates, and other unscheduled governance actions become unavailable.

Recommendations:

It is recommended to define an immutable, operationally meaningful maximum delay and enforce it in the constructor, `updateDelay()`, and `schedule()`.

```
// Example only: select the maximum through governance policy.
uint256 public constant MAX_DELAY = 365 days;

error DelayTooLong();

function _validateDelay(uint256 delay) internal pure {
    if (delay < MIN_DELAY) revert DelayTooShort();
    if (delay > MAX_DELAY) revert DelayTooLong();
}
```

```
constructor(
    uint256 _minDelay,
    address proposer,
    address executor,
    address canceller,
    address admin
) {
    _validateDelay(_minDelay);
    minDelay = _minDelay;
```

```
    // Existing role initialization
}

function schedule(
    address target,
    uint256 value,
    bytes calldata data,
    bytes32 predecessor,
    bytes32 salt,
    uint256 delay
) external onlyRole(PROPOSER_ROLE) {
    _validateDelay(delay);
    if (delay < minDelay) revert DelayTooShort();

    // Existing scheduling logic
}

function updateDelay(uint256 newDelay) external {
    require(
        msg.sender == address(this),
        "DaimonTimelock: only via self-call (governance proposal)"
    );

    _validateDelay(newDelay);

    emit MinDelayChanged(minDelay, newDelay);
    minDelay = newDelay;
}
```

The selected maximum should reflect the longest acceptable governance delay rather than merely avoiding arithmetic overflow.

Daimon DAO: Resolved with `@a70fd2b ...` — MAX_DELAY = 90 days, enforced in the constructor, updateDelay() and schedule().

Zenith: Verified.

[L-7] Permissionless pair pre-creation can deny token proxy initialization

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Medium

Target

- src/DaimonV2.sol#L43-L45
- src/DaimonV2.sol#L256-L257
- script/Deploy.s.sol#L82-L103

Description:

`DaimonV2.initialize()` unconditionally creates the canonical DMN/WBNB pair:

```
uniswapV2Router = IUniswapV2Router02(_router);
uniswapV2Pair = IUniswapV2Factory(uniswapV2Router.factory())
    .createPair(address(this), uniswapV2Router.WETH());
```

Canonical PancakeSwap V2 factories do not require token addresses to contain deployed code when `createPair()` is called. However, pair creation reverts if the canonical pair already exists.

The deployment script creates the proxy using `CREATE`, making its future address deterministic from the deployer address and nonce. An observer can predict that address and create its DMN/WBNB pair before the proxy transaction executes. The proxy constructor then delegates to `initialize()`, where `createPair()` reverts and rolls back the entire proxy deployment.

No uninitialized proxy remains, the attacker gains no control over the factory-created pair, and no token supply is lost. Nevertheless, an unprivileged attacker can disrupt deployment. Under the current broadcast sequence, a failed proxy transaction also consumes the deployer nonce, requiring the nonce-based Migration address prediction and subsequent deployment sequence to be recomputed. Repeating the attack requires predicting each replacement address and paying pair-creation gas each time.

The existing testnet deployment cannot be affected retroactively, but the issue remains relevant to the planned mainnet deployment.

Recommendations:

It is recommended to extend the factory interface with `getPair()` and reuse the trusted factory's canonical pair when one already exists:

```
interface IUniswapV2Factory {
    function getPair(address tokenA, address tokenB)
        external
        view
        returns (address pair);

    function createPair(address tokenA, address tokenB)
        external
        returns (address pair);
}
```

```
IUniswapV2Factory factory =
    IUniswapV2Factory(uniswapV2Router.factory());
address weth = uniswapV2Router.WETH();

address pair = factory.getPair(address(this), weth);
if (pair == address(0)) {
    pair = factory.createPair(address(this), weth);
}

uniswapV2Pair = pair;
```

Protect the subsequent initial-liquidity operation and account for any pre-existing pair balances or reserves. Private transaction submission can reduce reactive mempool front-running, but it is only defense in depth because a known deployer and nonce remain predictable.

Consider also decoupling the Migration address from a fragile sequence of consecutive deployer nonces, although this does not replace the pair-handling fix.

Daimon DAO: Resolved with `@c29eea54ca ...`

Zenith: Verified.

[L-8] Governor and Timelock cancellation state can diverge

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- src/DaimonGovernor.sol#L124-L217
- src/DaimonTimelock.sol#L84-L118

Description:

Governor and Timelock maintain independent cancellation state without synchronizing it.

Governor cancellation sets only the proposal flag:

```
function cancel(
    uint256 id
)
    external
    onlyGuardian
{
    Proposal storage p = proposals[id];

    if (p.executed) {
        revert AlreadyExecuted();
    }

    p.canceled = true;
    emit ProposalCanceled(id);
}
```

If the proposal has already been queued, its Timelock operation remains scheduled.

With the deployed role configuration, only Governor holds `EXECUTOR_ROLE`. Governor refuses to execute a canceled proposal, so the orphaned operation is not currently executable through the normal path.

Timelock permits governance to grant `EXECUTOR_ROLE` to additional addresses. Under that future configuration, an additional executor can call `Timelock.execute()` directly and execute the still-scheduled operation even though Governor reports the proposal as canceled.

Cancellation in the opposite direction also diverges. `Timelock.cancel()` modifies only its operation state:

```
function cancel(
    bytes32 id
)
    external
    onlyRole(CANCELLER_ROLE)
{
    Operation storage op = operations[id];

    require(
        !op.executed,
        "DaimonTimelock: already executed"
    );

    op.canceled = true;
    emit Cancelled(id);
}
```

Governor does not inspect this flag. A Timelock-canceled proposal retains `p.queued == true` and continues to report `Succeeded`, but every execution attempt reverts at Timelock.

Missing lifecycle validation creates additional inconsistencies:

- Governor accepts cancellation of nonexistent and future proposal IDs.
- When a pre-canceled ID is later created, `propose()` overwrites its proposal data but does not clear `p.canceled`.
- `castVote()` accepts votes on an existing canceled proposal during its original voting window.
- Timelock accepts cancellation of unscheduled and already-canceled operation IDs.
- A later `schedule()` overwrites an unscheduled cancellation because its `readyTimestamp` remains zero.

Under the current role configuration, the immediate effects are misleading state, wasted votes and transactions, permanently pre-canceled future proposals, and unreliable monitoring or incident response.

The higher-impact consequence: execution of an operation that Governor reports as canceled; this requires an explicit governance decision to grant another address direct Timelock execution authority.

Recommendations:

It is recommended to establish Governor as the authoritative cancellation path.

Require proposal existence and an explicitly cancellable state:

```
if (id >= proposalCount) {
    revert UnknownProposal();
}

Proposal storage p = proposals[id];

if (p.canceled || p.executed) {
    revert InvalidProposalState();
}
```

Reject votes on canceled proposals.

For queued proposals, derive the Timelock operation ID and cancel it atomically before finalizing Governor cancellation. Grant `CANCELLER_ROLE` to Governor while retaining the guardian's direct Timelock cancellation role as a time-limited emergency path; require both guardian cancellation paths to share the same immutable expiry, and ensure `state()` reflects direct Timelock cancellation so both contracts agree on executability.

```
if (p.queued) {
    bytes32 operationId =
        timelock.hashOperation(
            p.target,
            p.value,
            p.data,
            bytes32(0),
            p.timelockSalt
        );

    timelock.cancel(operationId);
}

p.canceled = true;
```

Restrict Timelock cancellation to operations that are scheduled, unexecuted, and not already canceled:

```
if (
    op.readyTimestamp == 0 ||
    op.executed ||
    op.canceled
) {
    revert InvalidOperationState();
}
```

Update Governor `state()` to:

- return `Queued` for a scheduled operation;
- return `Canceled` when either synchronized cancellation flag is set; and
- reject nonexistent proposal IDs.

If non-Governor proposers or executors are introduced later, define how their operations participate in the same authoritative cancellation model.

Governor and Timelock are not upgradeable. Apply these changes before mainnet through corrected deployments. Replacing Timelock after deployment requires particular care because other contracts may store its address immutably.

Add tests covering future and nonexistent IDs, voting after cancellation, cancellation before and after queueing, direct Timelock cancellation, repeated cancellation, scheduling after an invalid pre-cancel attempt, additional authorized executors, and consistent state across both layers.

Daimon DAO: Resolved with `@a0bd5d4897...`, `@7adfd637b1...`, `@5e2a4bc8340...`, and `@bd5c042f21...`

Zenith: Verified.

[L-9] Migration instructions exempt the wrong address from predecessor-token fees

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- src/DaimonMigration.sol#L4-L17
- src/DaimonMigration.sol#L91-L108
- script/Deploy.s.sol#L69-L77

Description:

The `DaimonMigration` header instructs operators to exempt the Migration contract from the predecessor token's transfer fees before opening migration.

```
// Operational precondition:
// oldDaimon.excludeFromFee(
//     address(thisMigrationContract)
// )
```

Migration is only the allowance spender. The actual old-token transfer moves DMX directly from the claimant to the immutable treasury:

```
bool ok = oldDaimon.transferFrom(
    msg.sender,
    treasury,
    amount
);
```

The verified predecessor token at `0x36EbA94407B53c631eE822C219e94580fadd67c7` passes only `sender` and `recipient` into its fee-bearing transfer path and disables fees only when one of those endpoints is exempt:

```
if (
    _isExcludedFromFee[from] ||
    _isExcludedFromFee[to]
) {
```

```
    takeFee = false;
}
```

Because Migration is neither endpoint, exempting it does not disable fees on claimant-to-treasury transfers. For the verified predecessor, the scalable address-based configuration is to exempt treasury. Exempting every claimant or setting the predecessor's fees to zero would also prevent the fee.

When neither claimant nor treasury is exempt, treasury receives less than the requested amount. Migration detects this and reverts:

```
uint256 actuallyReceived =
    treasuryBalanceAfter -
    treasuryBalanceBefore;

if (actuallyReceived != amount) {
    revert AmountMismatch();
}
```

The revert is atomic: the claimant retains their old DMX and receives no new DMN. The exact-delta check therefore protects users from silent underpayment, but claims from ordinary non-exempt holders remain unavailable until the predecessor configuration is corrected.

The testnet mock deployment correctly exempts treasury, but only in the branch that deploys `MockOldDaimon`. When a real `OLD_DAIMON` address is supplied for mainnet, the script neither performs nor verifies the required external configuration. `CHECKLIST_MAINNET.md` also omits this precondition.

Because `migrationDeadline` begins at deployment and is immutable, delayed discovery consumes the migration window. If the predecessor configuration cannot be corrected before expiry, affected holders lose access to the current migration round.

This issue concerns the old-DMX transfer into treasury. It is separate from DaimonV2's exemption of Migration for outgoing new-DMN claims.

Recommendations:

It is recommended to correct the source instructions and mainnet checklist to require exemption of treasury before deploying Migration.

```
oldDaimon.excludeFromFee(treasury);
```

Before deployment:

- verify the live predecessor's `transferFrom()` and fee-exemption semantics;
- verify that the current predecessor owner still has authority to exempt treasury or set fees to zero;
- execute and record the required configuration transaction;
- query treasury's exemption status if the predecessor exposes a suitable getter; and
- simulate or test a representative transfer from a non-exempt holder to treasury, confirming exact receipt.

Use a predecessor-specific deployment preflight rather than assuming a generic interface can prove fee behavior.

Do not begin the immutable migration deadline until the precondition has been confirmed. If the predecessor can no longer exempt treasury or disable fees, treat the current Migration design as incompatible and redesign and audit the migration flow before deployment.

Retain the exact balance-delta check. Removing it would convert a recoverable configuration outage into a silent violation of the promised 1:1 exchange.

Daimon DAO: Resolved with `@29ed47bc96 ...`

Zenith: Verified.

[L-10] Unbounded reward index can overflow per-account reward accounting

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- src/DaimonStaking.sol#L142-L200
- src/DaimonStaking.sol#L246-L281

Description:

`notifyRewardAmount()` is permissionless and increases a persistent global reward index. Because `stake()` accepts any nonzero amount, one wei staked under the default 1× option creates one voting-power unit.

While that account is the sole staker, every wei notified increases the index by `1e27`:

```
rewardPerVotingPowerStored +=
    (toDistribute * REWARD_PRECISION) /
    totalVotingPower;
```

The sole staker can immediately claim the notified BNB and reuse it in another notification. The same temporary BNB can therefore increase the index repeatedly, limited primarily by available liquidity, gas, and iteration count rather than permanent funding.

Reward accounting later multiplies an account's voting power by the accumulated index before dividing:

```
uint256 accumulated =
    (vp * rewardPerVotingPowerStored) /
    REWARD_PRECISION;
```

This checked intermediate multiplication is used when:

- initializing reward debt after staking;
- settling rewards before staking, withdrawal, or claiming;
- resetting reward debt after withdrawal; and
- calculating `pendingReward()`.

For voting power `v` and cumulative notifications of `C` BNB wei while the denominator is one, overflow occurs approximately when:

```
v × C × 1e27 > type(uint256).max
```

A future stake above the resulting arithmetic ceiling reverts atomically, including its token transfer, so that failed attempt does not lose principal.

An existing position can be affected through a narrower staged sequence:

1. The attacker pre-inflates and refines the index while remaining the sole staker.
2. A large user successfully stakes just below the arithmetic ceiling.
3. The attacker increases the index further.
4. The user's subsequent reward settlement overflows, blocking reward claims, additional staking, and withdrawal of principal.

The deployment-default `maxTxAmount` permits a single stake of up to 5 billion DMN. At the default 4× multiplier, this produces 20 billion DMN of voting power. Lowering the safe ceiling to that level requires approximately `5,789.6 BNB` of cumulative recycled notifications, assuming the index starts at zero and the attacker initially has one voting-power unit. This figure represents cumulative churn, not permanent expenditure, and `maxTxAmount` can later be changed by governance.

By comparison, lowering the ceiling below the deployed 1,000-DMN proposal threshold would require approximately `115.8 billion BNB` of cumulative churn. A practical system-wide governance shutdown is therefore not established. Existing eligible stakers can also continue proposing because governance reads voting power without invoking reward accounting.

The realistic impact is limited to denial or potential trapping of unusually large per-account positions. It requires the attacker to establish the favorable sole-staker state before honest staking and prepare a ceiling suitable for a future victim.

Recommendations:

It is recommended to use full-precision multiplication and division for both index updates and every per-account reward calculation.

```
import {Math} from
    "@openzeppelin/contracts/utils/math/Math.sol";

function _accumulatedReward(
    uint256 vp
)
```

```
    private
    view
    returns (uint256)
{
    return Math.mulDiv(
        vp,
        rewardPerVotingPowerStored,
        REWARD_PRECISION
    );
}
```

Use `_accumulatedReward()` consistently in `stake()`, `_settleReward()`, `withdraw()`, and `pendingReward()`. The index update should similarly use:

```
rewardPerVotingPowerStored += Math.mulDiv(
    toDistribute,
    REWARD_PRECISION,
    totalVotingPower
);
```

As defense in depth, prevent permissionless recycling under a dust-sized denominator. If public reward funding is not required, restrict `notifyRewardAmount()` to DaimonV2. Otherwise, define a minimum active voting-power policy together with an explicit policy for queued rewards; a minimum alone can reproduce first-staker reward capture.

Avoid a simple accumulator cap that makes legitimate token-funded notifications revert after the cap is reached.

DaimonStaking is not upgradeable, and DaimonGovernor stores its Staking source immutably. Remediation therefore requires coordinated replacement and rewiring of both contracts. Existing locks require a separate withdrawal or migration plan.

Daimon DAO: Resolved with `@c98bc801ed ...`

Zenith: Verified.

[L-11] Mutable mandatory fee exemptions can corrupt staking accounting and disrupt migration

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- src/DaimonV2.sol#L355-L385
- src/DaimonV2.sol#L629-L638
- src/DaimonStaking.sol#L142-L203
- src/DaimonMigration.sol#L91-L142

Description:

DaimonStaking and DaimonMigration require fee exemption for correct accounting and transfer liveness. Deployment configures these exemptions, but governance can later remove either through the general-purpose exemption setter.

```
function setExcludedFromFee(
    address account,
    bool excluded
)
    external
    onlyRole(GOVERNANCE_ROLE)
{
    isExcludedFromFee[account] = excluded;
    emit ExcludedFromFeeSet(account, excluded);
}
```

There is no distinction between optional user exemptions and exemptions required by active protocol modules.

If Staking's exemption is removed while fees are nonzero, transfers involving ordinary non-exempt users become taxable. `stake()` nevertheless credits the requested amount without measuring the balance received:

```
bool ok = daimonToken.transferFrom(
    msg.sender,
```

```
    address(this),
    amount
);
require(ok, "DaimonStaking: transferFrom failed");

uint256 vp =
    (amount * opt.multiplierX1000) / 1000;

totalStaked[msg.sender] += amount;
totalVotingPower += vp;
totalStakedAmount += amount;
```

The contract can therefore receive less than the recorded principal while granting voting power and BNB reward weight from the full requested amount. Reflection received by Staking may partially offset the shortfall but does not enforce exact backing. Repeated deposits can create a cumulative deficit, causing later withdrawals to consume tokens backing other users or revert.

Outgoing withdrawals can also deliver less than the recorded principal. Additionally, removing Staking's exemption makes it subject to `maxTxAmount` as a sender, so withdrawals above that limit revert outright.

Migration claims contain an exact recipient-delta check. If Migration's exemption is removed, an ordinary claim can either:

- revert immediately when its amount exceeds `maxTxAmount`; or
- be taxed and revert `AmountMismatch` because the claimant did not receive exactly 1:1.

The transaction rollback protects the user's old DMX, but migration becomes unavailable while the configuration remains incorrect. Because `migrationDeadline` is immutable, the remaining migration window can expire before governance restores the exemption.

The final sweep has two possible failure modes after exemption removal:

- if `remaining > maxTxAmount`, the transfer reverts and `sweepExecuted` rolls back atomically; or
- if `remaining ≤ maxTxAmount` and neither endpoint is exempt, the taxed transfer can return `true` while treasury receives less than `remaining`.

In the second case, `sweepExecuted` remains permanently set despite the treasury shortfall, preventing another sweep. Part of the amount is instead allocated through token fees and reflection, with possible residual rounding dust.

These effects require nonzero fees and non-exempt counterparties. They are not reachable by an unprivileged caller because exemption removal requires a successful governance vote and Timelock execution. Nevertheless, the permitted configuration change silently violates the documented 1:1 staking and migration assumptions.

Recommendations:

It is recommended to distinguish mandatory protocol exemptions from discretionary exemptions and prevent removal while the corresponding module can still hold liabilities or perform transfers.

```
mapping(address => bool)
    public mandatoryFeeExempt;

error MandatoryFeeExemption();

function setExcludedFromFee(
    address account,
    bool excluded
)
    external
    onlyRole(GOVERNANCE_ROLE)
{
    if (
        !excluded &&
        mandatoryFeeExempt[account]
    ) revert MandatoryFeeExemption();

    isExcludedFromFee[account] = excluded;
    emit ExcludedFromFeeSet(account, excluded);
}
```

Mark Migration and every Staking contract with outstanding locks as mandatory. When rotating Staking, protect the new contract without removing protection from the previous contract until `totalStakedAmount == 0`. Migration protection should remain at least until the deadline has passed and an exact final sweep has completed.

For the upgradeable token, append any new storage variables safely and initialize protection for existing module addresses.

Add exact balance-delta checks as defense in depth:

- `stake()` must receive exactly the recorded principal;
- `withdraw()` must deliver exactly the recorded principal;
- `sweepUnclaimed()` must verify the treasury's exact balance increase and the final Migration balance; and
- `sweepExecuted` should be finalized only after these postconditions hold.

These checks prevent silent accounting loss, while mandatory exemption protection preserves protocol liveness.

Daimon DAO: Resolved with `@b0ea5544f6 ...` and `@c98bc801ed ...` The token preserves mandatory exemptions while liabilities exist, and Staking structurally rejects new deposits

whenever it is not fee-exempt.

Zenith: Verified.

[L-12] Spot-derived minimum output does not provide the claimed MEV-loss bound

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- src/DaimonV2.sol#L369-L381
- src/DaimonV2.sol#L487-L508
- src/DaimonV2.sol#L511-L548

Description:

Fee swaps and buybacks derive `amountOutMin` from PancakeSwap's current reserve-based quote immediately before executing the swap.

```
uint256[] memory quote =
    uniswapV2Router.getAmountsOut(tokenAmount, path);

uint256 minOut =
    (quote[1] * (10000 - maxSwapSlippageBps)) /
    10000;
```

The buyback follows the same pattern after adjusting the quoted output for token transfer fees.

```
uint256 expectedAfterFee =
    (quote[1] *
        (1000 - taxFee - liquidityFee)) /
    1000;

uint256 minOut =
    (expectedAfterFee *
        (10000 - maxSwapSlippageBps)) /
    10000;
```

`maxSwapSlippageBps` correctly limits output relative to the immediate PancakeSwap quote. It does not limit economic loss relative to a fair, pre-attack, or time-weighted price because an attacker can manipulate the pair's reserves before the quote is read. Both the quote and

minimum output then use the manipulated price.

The quote and swap occur atomically, with no external transaction able to update reserves between them. On the one-hop PancakeSwap V2 paths used here, the check therefore normally accepts the reserve state it just observed. The `try/catch` blocks preserve the triggering user transfer if a swap fails, but they do not reject a successful swap executed at a manipulated price.

Balances, thresholds, operation sizes, and buyback parameters are public. Any funded holder can trigger processing by transferring tokens to the pair once the relevant conditions are met.

Buybacks can be targeted by raising the token's spot price before triggering the operation and unwinding afterward. This is clearest when the contract holds sufficient BNB while its token balance remains below the fee-swap threshold. When a fee swap also precedes the buyback, that deterministic operation can be included in the attacker's strategy.

Manipulating token-to-BNB fee swaps requires staging because an attacker's sale made while the threshold is already met triggers the protocol swap before the attacker's transfer reaches the pair. However, the threshold is checked before fees from the triggering transfer are collected. An attacker can:

1. Sell while the pre-transfer fee balance is below the threshold.
2. Depress the reserves while the sale's fees push the balance above the threshold.
3. Trigger processing with another transfer while the manipulated reserve state remains effective.
4. Unwind after the protocol swap.

The preparation and trigger calls can be bundled within one attacker-controlled transaction where available.

Consequently, loss measured against an independent fair-price reference can materially exceed `maxSwapSlippageBps`. Profitability depends on liquidity, protocol operation size, AMM fees, gas, manipulation cost, and competing MEV.

The documentation acknowledges same-block MEV as an accepted limitation but states that damage remains within the configured tolerance. That statement is only true relative to the contemporaneous router quote, not relative to an unmanipulated market price.

Recommendations:

It is recommended to validate the current spot price against an independent, delayed reference before executing fee swaps or buybacks.

Use a cumulative-price TWAP or robust oracle with:

- a manipulation-resistant observation window;

- explicit staleness checks;
- a maximum permitted spot/reference deviation; and
- safe behavior when reference data is unavailable.

If the deviation exceeds the configured limit, skip the protocol operation without reverting the user transfer that triggered processing.

Derive `amountOutMin` from a conservative expected output that accounts for the reference price, permitted AMM price impact, token transfer fees, and acceptable oracle deviation. A TWAP represents a reference price and should not be treated as the exact output quote for a large swap.

Limit each operation relative to available pool liquidity and process accumulated balances in smaller slices. This reduces price impact and manipulation profitability but does not replace an independent price reference.

If the protocol deliberately chooses not to use an oracle, document that `maxSwapSlippageBps` protects only against deviation from the immediate router quote and does not bound total MEV loss relative to fair price. In that case, conservatively cap operation sizes and treat the remaining exposure as an explicitly unbounded accepted risk.

Add tests covering:

- reserve manipulation before fee swaps and buybacks;
- fee-balance threshold crossing during the attacker's preparatory sale;
- spot/TWAP deviation rejection;
- stale or unavailable reference data; and
- preservation of the triggering user transfer when processing is skipped.

Daimon DAO: Resolved with `@41b4d891e6 ...` by correcting the documentation to state that `maxSwapSlippageBps` only bounds deviation from the contemporaneous router quote, not loss relative to a fair price. The protocol intentionally uses no oracle, and the remaining MEV exposure is explicitly accepted as unbounded.

Zenith: Verified.

4.5 Informational

A total of 16 informational findings were identified.

[I-1] Staking assumes deposits are received in full

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- DaimonStaking.sol

Description:

`stake()` checks that `daimonToken.transferFrom()` returns true, then records the requested `amount` in `Lock.amount`, `totalStaked`, and `totalStakedAmount`. It also calculates voting power from that nominal amount without checking the staking contract's actual balance increase.

The intended integration satisfies this assumption because `DaimonV2.setStakingContract()` marks the staking contract as exempt from transfer fees. Deposits should therefore deliver the exact requested amount under the current configuration.

However, the staking contract does not document that exact, fee-free receipt is an accounting invariant. If the exemption is removed or future token behavior deducts an amount during transfers, staking would record more principal and voting power than it received, leaving its liabilities greater than its token balance.

Recommendations:

Document in `DaimonStaking` and the NatSpec for `stake()` that `daimonToken.transferFrom()` must credit exactly `amount` to the staking contract. State that the staking address must be configured and remain fee-exempt before deposits are accepted, and that token upgrades or configuration changes must preserve this invariant.

Daimon DAO: Resolved with `@41b4d89 ...`

Zenith: Verified.

[I-2] A taxed final migration sweep permanently closes recovery after an incomplete transfer

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- DaimonMigration.sol

Description:

`claim()` protects its 1:1 guarantee with recipient balance-delta checks, but `sweepUnclaimed()` checks only the boolean return from `newDaimon.transfer()`.

Governance can revoke the migration contract's fee exemption before the deadline. In that state, the final transfer succeeds while the treasury receives less than `remaining`, the difference is redirected through reflection and the fee bucket. Because `sweepExecuted` remains true after this successful taxed transfer, the migration's recovery operation cannot be retried after correcting the configuration.

The emitted `Swept` event also reports the nominal amount rather than what the treasury received.

Recommendations:

Measure the treasury balance before and after the transfer and require an exact increase. A mismatch should revert the whole transaction, preserving the ability to restore the exemption and retry.

Daimon DAO: Resolved with `@700146f ...` — sweepUnclaimed() now measures the treasury balance before and after and requires an exact increase, reverting on any shortfall. Same delta logic already present in claim().

Defence in depth on top of #32, which now prevents the Migration's fee exemption being removed while the deadline hasn't passed and the sweep hasn't executed.

Zenith: Verified.

[I-3] Caught router failures leave residual DMN allowances

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- src/DaimonV2.sol#L487-L508

Description:

`_swapTokensForEth()` approves the router before invoking it inside a swallowed `try/catch`:

```
_approve(
    address(this),
    address(uniswapV2Router),
    tokenAmount
);

try uniswapV2Router
    .swapExactTokensForETHSupportingFeeOnTransferTokens(
        tokenAmount,
        minOut,
        path,
        address(this),
        block.timestamp
    )
{} catch {}
```

If the router call reverts, state changes made inside that external-call subtree, including any `transferFrom()` and allowance consumption, are rolled back. The approval was written in DaimonV2's successful outer execution frame and therefore remains equal to `tokenAmount` after the caught failure.

A later successful attempt normally overwrites and consumes the allowance. However, it can persist indefinitely when no later attempt succeeds. In particular, subsequent `getAmountsOut()` failures occur before the next approval and therefore do not clear the existing allowance.

The canonical PancakeSwap V2 router does not let arbitrary callers instruct it to spend from DaimonV2, so no current unprivileged drain path was identified. The residual approval

nevertheless violates the intended zero-allowance postcondition and increases dependency exposure if:

- an incompatible router is configured;
- the trusted router develops an unexpected vulnerability; or
- a future token upgrade changes router interaction or permits router rotation.

The reviewed implementation exposes no router setter, which limits this to defense-in-depth dependency risk.

Recommendations:

It is recommended to clear any existing router allowance during the upgrade that introduces the fix. Runtime swap logic should then maintain a zero allowance outside the router call.

Clear the allowance before quoting, catch quote failures, grant only the required allowance, and revoke it after either swap outcome:

```
_approve(
    address(this),
    address(uniswapV2Router),
    0
);

uint256 minOut;

try uniswapV2Router.getAmountsOut(
    tokenAmount,
    path
) returns (uint256[] memory quote) {
    require(
        quote.length >= 2,
        "DaimonV2: malformed quote"
    );

    minOut =
        (quote[1] *
            (10000 - maxSwapSlippageBps)) /
        10000;
} catch {
    return;
}

_approve(
    address(this),
    address(uniswapV2Router),
```

```
    tokenAmount
);

try uniswapV2Router
    .swapExactTokensForETHSupportingFeeOnTransferTokens(
        tokenAmount,
        minOut,
        path,
        address(this),
        block.timestamp
    )
{} catch {}

_approve(
    address(this),
    address(uniswapV2Router),
    0
);
```

Any future router-rotation function should revoke the old router's allowance before changing the stored address.

Add integration tests covering successful swaps, caught swap failures, caught quote failures, a router that returns without consuming its full allowance, upgrade cleanup of an existing allowance, and router rotation.

Daimon DAO: Resolved with `@e5cb1f4009 ...`

Zenith: Verified.

[I-4] Staking governance changes lack a dedicated event

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- src/DaimonStaking.sol#L34-L52
- src/DaimonStaking.sol#L311-L313

Description:

`DaimonStaking` stores authorized governance accounts in a private, non-enumerable mapping. `_setGovernance()` changes this mapping without emitting an event:

```
function _setGovernance(address account, bool enabled) internal {
    _governance[account] = enabled;
}
```

Both the initial governance assignment and subsequent calls to `setGovernance()` therefore lack a dedicated contract-level record of the affected account and its new authorization state.

The changes are not invisible: monitors can decode transaction calldata, inspect generic Timelock events, or query `isGovernance()` for known addresses. However, they cannot subscribe to a dedicated staking-governance event or enumerate the mapping to discover previously unknown authorized accounts.

This does not bypass authorization or affect protocol state correctness. It increases the complexity and reduces the reliability of privilege monitoring and incident-response tooling.

Recommendations:

It is recommended to emit an indexed event from `_setGovernance()` so initial and subsequent changes are recorded consistently. Avoid emitting an event when the requested value does not change state:

```
event GovernanceSet(
    address indexed account,
    bool enabled,
```

```
    address indexed caller
);

function _setGovernance(address account, bool enabled) internal {
    if (_governance[account] == enabled) return;

    _governance[account] = enabled;
    emit GovernanceSet(account, enabled, msg.sender);
}
```

Add tests covering the initial assignment, enabling an account, disabling an account, and no-op updates.

Daimon DAO: Resolved with `@3a7331863e ...`

Zenith: Verified.

[I-5] Timelock accepts cancellation of nonexistent operations

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- DaimonTimelock.sol

Description:

`cancel()` checks only that an operation has not executed. An unknown identifier resolves to an empty `Operation`, passes that check, is marked canceled, and emits `Cancelled(id)` even though it was never scheduled.

The cancellation does not reserve or permanently block the identifier. `schedule()` checks only whether `readyTimestamp` is nonzero, then overwrites the full structure with `canceled: false`. A later operation with the same identifier can therefore be scheduled normally despite the earlier cancellation event.

This can mislead monitoring systems and allows a guardian typo or incorrect identifier calculation to appear successful. Unlike a revert, the emitted event gives operators no indication that the intended scheduled operation remains active.

Recommendations:

Require the operation to have a nonzero `readyTimestamp` and to be in a cancellable scheduled state before setting `canceled`. Revert for unknown, already canceled, or already executed identifiers.

Daimon DAO: Resolved with `@a0bd5d4 ...` — cancel() now reverts for unscheduled operations (OperationNotScheduled) and for already-canceled ones (OperationAlreadyCanceled), before writing any state.

Zenith: Verified.

[I-6] Unswapped fee inventory adopts the current marketing-to-buyback ratio

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- src/DaimonV2.sol#L410-L430
- src/DaimonV2.sol#L459-L470
- src/DaimonV2.sol#L580-L586

Description:

Fee-bearing transfers combine the buyback and marketing fees into `liquidityFee`. The resulting tokens are credited to the contract as one balance, without recording their intended branch:

```
uint256 tLiquidity = (tAmount * liquidityFee) / 1000;

if (tLiquidity > 0) _takeLiquidity(tLiquidity);
```

When accumulated tokens are later swapped, the received BNB is divided using the current marketing-to-buyback ratio:

```
uint256 marketingEth = (ethReceived * marketingFee) / liquidityFee;
// The remainder stays in the contract for buyback.
```

Governance can change this ratio through `setFees()` while tokens collected under the previous ratio remain unswapped. A subsequent qualifying sale therefore distributes the old inventory using the new ratio. If `liquidityFee` has been changed to zero, a successful swap occurs before the zero-value check and all proceeds remain in the contract, where an enabled buyback can later use them.

Only governance can create this condition, and the change is subject to the Timelock. There is no unprivileged theft or direct loss. The impact is that previously collected fees may be attributed differently from the parameters in effect when they were collected.

Whether this is undesirable depends on the intended policy: accrual-time allocation preserves the original fee destination, while execution-time allocation deliberately applies cur-

rent governance settings to all unswapped inventory.

Recommendations:

It is recommended to define the intended allocation policy explicitly.

If accrual-time allocation is required, record separate pending marketing and buyback token balances when fees are collected. Each swap should consume and decrement those balances and divide the resulting BNB according to the inventory actually consumed. Reflected or directly donated surplus should use a separate documented allocation policy.

If execution-time allocation is intentional, document that changes to the fee split apply to all unswapped inventory; no accounting change is then required.

Daimon DAO: Resolved with `@41b4d891e6...` and `@e649907c23...` by explicitly adopting execution-time allocation and correcting the documentation to state that each swap converts one threshold-sized tranche while total unswapped inventory can exceed that threshold.

Zenith: Verified.

[I-7] Zero-value transfers and token events are not fully standards-compatible

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- src/DaimonV2.sol#L263-L318
- src/DaimonV2.sol#L355-L435
- src/DaimonV2.sol#L551-L575

Description:

`_transfer()` rejects zero-value transfers:

```
if (from == address(0) || to == address(0)) revert ZeroAddress();
require(amount > 0, "DaimonV2: transfer amount is zero");
```

ERC-20 and BEP-20 require zero-value transfers to be treated as normal transfers and emit a `Transfer` event. Integrations relying on this behavior can therefore encounter unexpected reverts. Transfers remain intentionally unavailable while the token is paused, including zero-value transfers.

Taxed transfers emit only the net amount received by the destination:

```
if (tLiquidity > 0) _takeLiquidity(tLiquidity);
if (rFee > 0 || tFee > 0) _reflectFee(rFee, tFee);

emit Transfer(sender, recipient, tTransferAmount);
```

The explicit fee credited to the token contract has no corresponding `Transfer` event. Reflection also changes holder balances through the global conversion rate without per-holder events. This is inherent to the reflection design, but it means event-only accounting cannot reconstruct all balances.

`burnDeadBalanceToFloor()` reduces both the dead-address balance and `totalSupply` without emitting the conventional supply-reduction event:

```
Transfer(deadAddress, address(0), toBurn)
```

Finally, the token does not implement the `getOwner()` extension required for strict BEP-20/BEP-2 binding compatibility. This method is not part of ERC-20 and is unnecessary for ordinary BSC transfers. The protocol deliberately uses role-based governance without a singular owner, so providing a single owner address may not accurately represent its authority model.

On-chain balances remain authoritative, and these differences do not create or lose value. Their impact is limited to compatibility with wallets, explorers, indexers, bridges, exchanges, and accounting systems that rely on strict interfaces or event reconstruction.

Recommendations:

It is recommended to support zero-value transfers with an early return before swap and buyback automation:

```
if (from == address(0) || to == address(0)) revert ZeroAddress();

if (amount == 0) {
    emit Transfer(from, to, 0);
    return;
}
```

Emit a separate `Transfer` event for the fee credited to the token contract and a custom event for the reflection fee:

```
event ReflectionFeeApplied(
    address indexed sender,
    uint256 amount
);
```

```
emit Transfer(sender, recipient, tTransferAmount);

if (tLiquidity > 0) {
    emit Transfer(sender, address(this), tLiquidity);
}

if (tFee > 0) {
    emit ReflectionFeeApplied(sender, tFee);
}
```

Emit the conventional burn event after successfully updating the dead-address balance and

supply:

```
emit Transfer(deadAddress, address(0), toBurn);
```

Document that reflected balances cannot be reconstructed exclusively from events and that integrations must query `balanceOf()`.

If BEP-2 binding is required, define and maintain meaningful `getOwner()` semantics for the role-based governance model. Otherwise, explicitly document that this binding extension is unsupported rather than returning a misleading owner address.

Daimon DAO: Resolved with `@44728761a6 ...`

Zenith: Verified.

[I-8] Governor state omits queued status and treats unknown proposals as defeated

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- src/DaimonGovernor.sol#L56-L80
- src/DaimonGovernor.sol#L143-L208

Description:

`ProposalState` defines a `Queued` state, and each proposal stores a `queued` flag. However, `state()` never reads that flag or returns `ProposalState.Queued`:

```
enum ProposalState {
    Pending,
    Active,
    Defeated,
    Succeeded,
    Queued,
    Executed,
    Canceled
}
```

```
if (quorumVotes < quorumNeeded || p.forVotes <= p.againstVotes) {
    return ProposalState.Defeated;
}

return ProposalState.Succeeded;
```

A queued proposal that continues satisfying the success conditions therefore reports `Succeeded` until it is executed or canceled. Functional execution remains protected because `execute()` separately checks `p.queued`, while the public proposal getter exposes the flag. Nevertheless, integrations cannot rely on `state()` alone to distinguish successful unqueued proposals from queued proposals.

`state()` also does not validate that `id < proposalCount`. Storage for an unknown proposal

ID contains zero values. After the timestamp checks, `forVotes ⩽ againstVotes` evaluates as `0 ⩽ 0`, causing the unknown proposal to report `Defeated`.

These behaviors do not bypass voting or execution requirements. They can produce misleading lifecycle information for interfaces, monitoring systems, indexers, and operational tooling.

Recommendations:

It is recommended to reject unknown proposal IDs and return the existing `Queued` state after confirming that the proposal still satisfies the success conditions:

```
error UnknownProposal(uint256 id);

function state(uint256 id) public view returns (ProposalState) {
    if (id >= proposalCount) revert UnknownProposal(id);

    Proposal storage p = proposals[id];

    if (p.canceled) return ProposalState.Canceled;
    if (p.executed) return ProposalState.Executed;
    if (block.timestamp < p.voteStart) return ProposalState.Pending;
    if (block.timestamp <= p.voteEnd) return ProposalState.Active;

    uint256 quorumVotes = p.forVotes + p.abstainVotes;
    uint256 quorumNeeded =
        (p.snapshotTotalVotingPower * quorumBps) / 10000;

    if (
        quorumVotes < quorumNeeded ||
        p.forVotes <= p.againstVotes
    ) {
        return ProposalState.Defeated;
    }

    if (p.queued) return ProposalState.Queued;
    return ProposalState.Succeeded;
}
```

Update `execute()` to accept `Queued` while preserving meaningful errors:

```
Proposal storage p = proposals[id];

if (p.executed) revert AlreadyExecuted();
```

```
ProposalState currentState = state(id);

if (currentState == ProposalState.Succeeded) {
    revert ProposalNotQueued();
}

if (currentState != ProposalState.Queued) {
    revert ProposalNotSucceeded();
}
```

The separate `p.queued` check can then be removed. Apply proposal-existence validation consistently to other functions that accept a proposal ID.

Daimon DAO: Resolved with `@5e2a4bc ...` state() now returns Queued for scheduled proposals and reverts with UnknownProposal for ids >= proposalCount, instead of reporting an empty struct as Pending.

Zenith: Verified.

[I-9] Governance approvals do not expire automatically

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- src/DaimonGovernor.sol#L164-L208
- src/DaimonTimelock.sol#L84-L111

Description:

After voting ends, `DaimonGovernor.state()` does not impose a maximum validity period. A proposal that continues to satisfy the success conditions remains queueable indefinitely:

```
function queue(uint256 id) external {
    if (state(id) != ProposalState.Succeeded) {
        revert ProposalNotSucceeded();
    }

    // No maximum interval since voteEnd
}
```

Queueing an old proposal starts a fresh Timelock delay, preserving the mandatory seven-day reaction window. Once the operation becomes ready, however, `DaimonTimelock.execute()` enforces only the minimum delay:

```
if (block.timestamp < op.readyTimestamp) {
    revert TooEarly();
}

// No upper execution deadline
```

A ready operation therefore remains executable until it is executed or canceled.

This does not authorize an action that governance did not approve, and repeated execution is prevented. Execution through the Governor also requires the proposal to remain successful, queued, and not canceled. The designated cancellation authority can cancel stale proposals and operations.

Nevertheless, an approved action may execute after contract balances, market conditions, governance participation, implementation assumptions, or community expectations have materially changed.

This behavior matches OpenZeppelin `TimelockController`, which defines a minimum delay without an execution deadline. It also avoids requiring an actor to execute within a fixed period. It should therefore be treated as an Informational governance-lifecycle design consideration rather than an implementation vulnerability.

Recommendations:

If indefinite validity is intentional, document that successful proposals and ready operations remain valid until executed or canceled. Operational monitoring should identify and cancel approvals that are no longer appropriate.

If bounded validity is desired, add:

- a queue grace period measured from the proposal's fixed `voteEnd`; and
- an execution grace period stored when the Timelock operation is scheduled.

```
if (block.timestamp > p.voteEnd + QUEUE_GRACE_PERIOD) {
    revert ProposalExpired();
}

if (block.timestamp > op.executionDeadline) {
    revert OperationExpired();
}
```

Store deadlines when the relevant lifecycle transition occurs rather than recomputing them from a later delay value. Expose expiration consistently in both contracts.

Expired actions should require a new proposal and operation identifier. The grace periods should be long enough to accommodate transient execution failures and avoid introducing excessive governance-liveness requirements.

Daimon DAO: Resolved with `@41b4d891e6 ...` and updated by `@bd5c042f21 ...` Indefinite queue and execution validity remains an accepted governance-lifecycle policy. During the 36-month guardian mandate, stale approvals are handled through monitoring and guardian cancellation; after that authority expires, a queued-but-unwanted operation can be canceled through a governance-voted Timelock self-call. Automatic expiry remains intentionally omitted to avoid forcing governance to repeat the full proposal cycle.

Zenith: Verified.

[I-10] EXECUTOR_ROLE cannot be opened as documented

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- DaimonTimelock.sol

Description:

The declaration of `EXECUTOR_ROLE` states that the role can be opened to anyone. However, `execute()` uses AccessControl's standard `onlyRole(EXECUTOR_ROLE)` modifier, which always checks the caller's concrete membership.

Granting `EXECUTOR_ROLE` to `address(0)` does not make execution permissionless under this modifier. OpenZeppelin's Timelock implements that configuration with a separate role-or-open check that accepts every caller when the zero address holds the role.

The current deployment grants the role to `DaimonGovernor`, whose public `execute()` function remains callable by anyone. There is no immediate liveness failure, but an operator following the Timelock comment during a future role rotation could leave no usable direct executor.

Recommendations:

Remove the claim that the role can be opened if only concrete executors are supported. If permissionless direct Timelock execution is intended, implement and test a role-or-open modifier before documenting `address(0)` as the open-role configuration.

Daimon DAO: Resolved with `@702b101 ...` — removed the claim that EXECUTOR_ROLE can be opened to everyone by granting it to address(0). execute() uses the standard only-Role() modifier, which always checks concrete membership, so only concrete executors are supported.

Zenith: Verified.

[I-11] A predecessor operation can appear as executed before its call finishes

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- DaimonTimelock.sol

Description:

`execute()` sets `op.executed = true` before calling the operation's target. During that external call, the Timelock therefore reports the operation as fully executed even though its payload is still running.

If the target can call `execute()` because it also holds `EXECUTOR_ROLE`, it can reenter with a second ready operation whose `predecessor` is the first operation. The predecessor check succeeds and the dependent payload runs in the middle of its predecessor rather than after the predecessor has completed. This can expose the dependent call to partially updated external state and violates the ordering promised by the predecessor field.

The deployed Governor is the sole executor and its queue path always uses a zero predecessor, so the required role and dependency configuration is not currently exposed. The behavior becomes reachable if governance later adds an executor that can also be an operation target and uses nonzero predecessors.

Recommendations:

Given that the current Governor never schedules operations with a nonzero predecessor, the protocol can accept this limitation and document that predecessor-based ordering is unsupported under the current execution model. If predecessor support is intended, adopt an approach like OpenZeppelin's `TimelockController`, which marks an operation done only after its external call completes and performs a post-call state check to catch reentrant state changes.

Daimon DAO: Resolved with `@41b4d89 ...`

Zenith: Verified.

[I-12] Emergency pause does not stop real supply burns

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- DaimonV2.sol

Description:

The `whenNotPaused` check is applied through `_transfer()`, but the permissionless `burnDeadBalanceToFloor()` entry point has no equivalent guard. Any account can therefore reduce `_tTotal`, `_rTotal`, and the dead address balance while the guardian has paused token transfers.

The function can only consume tokens already held by the dead address and cannot burn below `MIN_SUPPLY`, so it does not expose live user balances. It nevertheless makes an irreversible supply and reflection-accounting change during an emergency period in which operators may expect token state to remain stable while an incident is investigated.

The current documentation describes burning as permissionless at any time. The omission should therefore be resolved as an explicit pause-policy choice rather than left ambiguous.

Recommendations:

If pausing is intended to freeze all material token-state transitions, apply `whenNotPaused` to `burnDeadBalanceToFloor()`. Otherwise, document that supply burns deliberately remain available during a pause and include that behavior in incident procedures.

Daimon DAO: Resolved with `@d36cb5c ...` — whenNotPaused applied to burnDeadBalanceToFloor().

Zenith: Verified.

[I-13] Treasury custody permits repeated migration of legacy tokens

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- DaimonMigration.sol

Description:

`claim()` transfers legacy tokens to the treasury rather than burning or locking them. It verifies only that the treasury balance increased by the requested amount before paying the same amount of new DMN. `migratedAmount` records prior claims but does not limit later claims by the same account.

If the treasury transfers collected legacy tokens back to a claimant while migration remains open, the same units can be deposited again and exchanged for another allocation of new tokens. Repeating the cycle can consume new-token inventory that is intended to back legacy tokens still held by other users.

This requires cooperation, compromise, or an operational mistake by the fixed treasury, so it is primarily a custody and migration-policy risk. The contract does not enforce the assumption that collected legacy tokens remain out of circulation for the full migration window.

Recommendations:

Keep all collected legacy tokens in non-circulating custody until migration closes and document this as a strict treasury requirement. For on-chain enforcement, send them to an irrevocable burn address or a dedicated vault that cannot release them before the deadline.

Daimon DAO: Accepted as a custody risk. Resolved with `@41b4d89 ...` and added to CHECKLIST_MAINNET as an operational requirement — collected legacy tokens stay in non-circulating custody for the full migration window, and custody ownership is decided before the window opens. The base deadline is immutable, but the M-7 fix means the window it governs is not fixed: `cumulativePauseSeconds` extends the effective deadline by any time the token spends paused, so a guardian cannot consume the claim window. The custody requirement therefore runs to the effective deadline rather than the base one, and holding until `sweepUnclaimed()` has executed is the operationally safe reading.

Zenith: Verified.

[I-14] Duplicate queue attempts rely on Timelock reverts

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- DaimonGovernor.sol

Description:

`queue()` does not check `p.queued` before trying to schedule a proposal. Because `state()` continues to return `Succeeded` after the first queue, every later call reaches `DaimonTimelock.schedule()` with the same operation parameters and salt.

The Timelock currently rejects the repeated operation with `OperationAlreadyScheduled()`, which reverts the Governor's redundant assignment to `p.queued`. This prevents duplicate scheduling, but makes the Governor rely on a lower-level implementation detail instead of enforcing its own lifecycle.

The missing guard also returns a Timelock error for a Governor state violation and makes the behavior fragile if the scheduling implementation or operation identifier changes.

Recommendations:

Check `p.queued` before changing state or calling the Timelock and revert with a dedicated Governor error when a proposal has already been queued.

Daimon DAO: Resolved with `@6c64827 ...` — queue() now checks p.queued before touching state or calling the Timelock, reverting with ProposalAlreadyQueued().

Zenith: Verified.

[I-15] Guardian can permanently pre-cancel proposals that do not yet exist

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- DaimonGovernor.sol

Description:

`cancel()` does not verify that the referenced proposal exists:

```
Proposal storage p = proposals[id];
if (p.executed) revert AlreadyExecuted();
p.canceled = true;
```

Because mappings return an empty record for unknown IDs, the guardian can call `cancel(proposalCount)` or cancel an arbitrary range of future IDs.

When one of those IDs is later allocated, `propose()` populates individual fields but never resets `p.canceled`. The new proposal is therefore created in the canceled state. These stored cancellation flags persist even if the guardian is subsequently rotated or loses authority.

Recommendations:

Reject cancellation of nonexistent proposals and restrict cancellation to appropriate live states:

```
if (id >= proposalCount) revert ProposalDoesNotExist();
```

Daimon DAO: Resolved with `@7adfd63...` — cancel() now reverts with ProposalDoesNotExist() for id >= proposalCount and ProposalAlreadyCanceled() for already canceled proposals.

Zenith: Verified.

[I-16] Plain BNB transfers to staking are accepted but never accounted

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- DaimonStaking.sol

Description:

The unrestricted `receive()` function accepts BNB without updating `rewardPerVotingPowerStored` or `undistributedRewards`. No recovery function exists, so ordinary BNB sent directly rather than through `notifyRewardAmount()` remains outside all reward accounting.

Recommendations:

Have `receive()` account ordinary receipts using the same logic as reward notifications, or remove the function to prevent BNB transfers.

Daimon DAO: Resolved with `@7c3fe8a ...` — receive() removed.

Zenith: Verified.